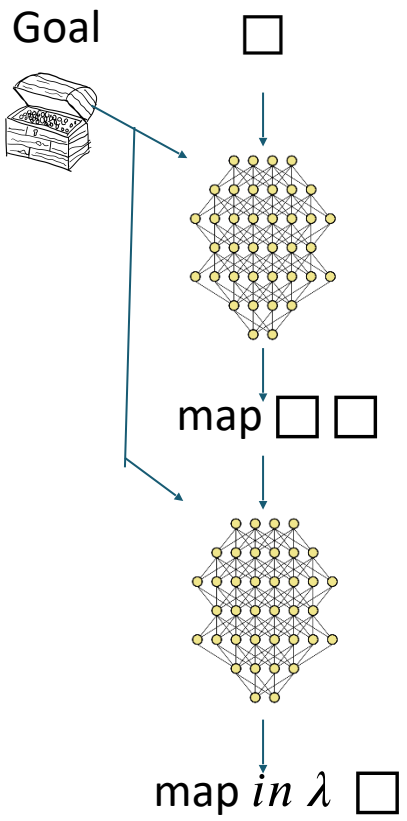
Lecture 24
Neurosymbolic Synthesis

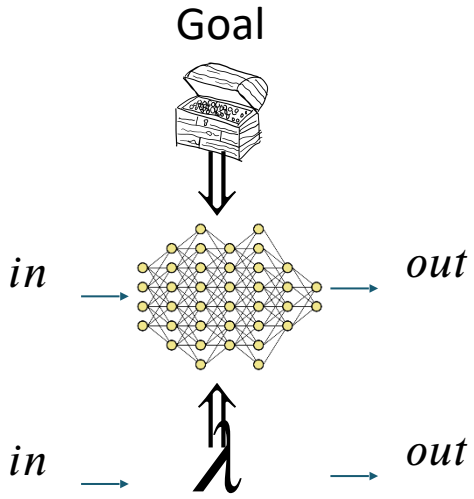
Armando Solar-Lezama with slides from Swarat Chaudhuri

Neurosymbolic program synthesis

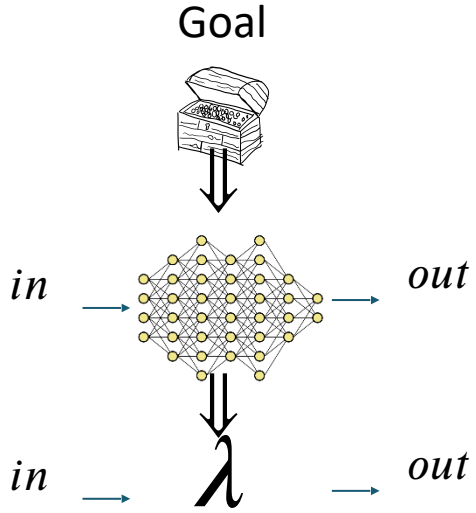
Neural Guided Search



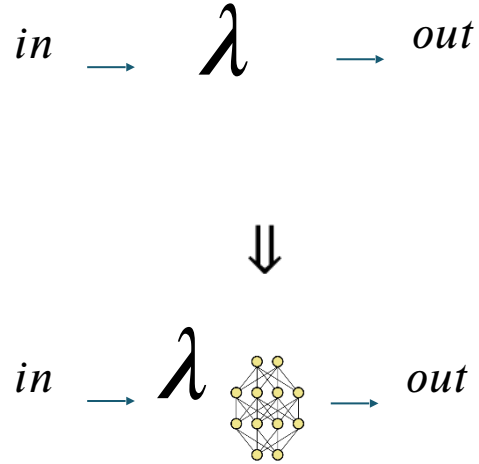
Symbolic Guided DL



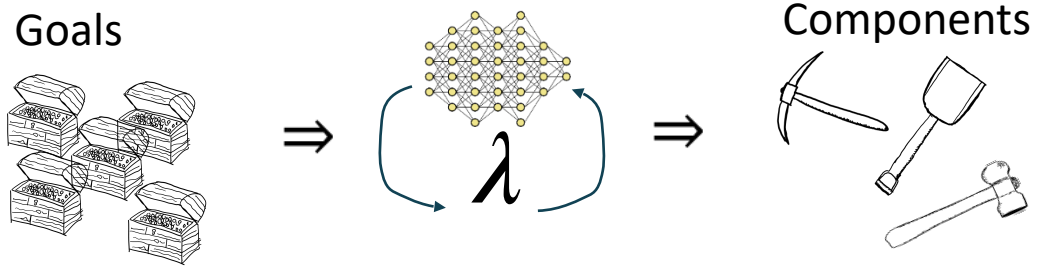
Distillation



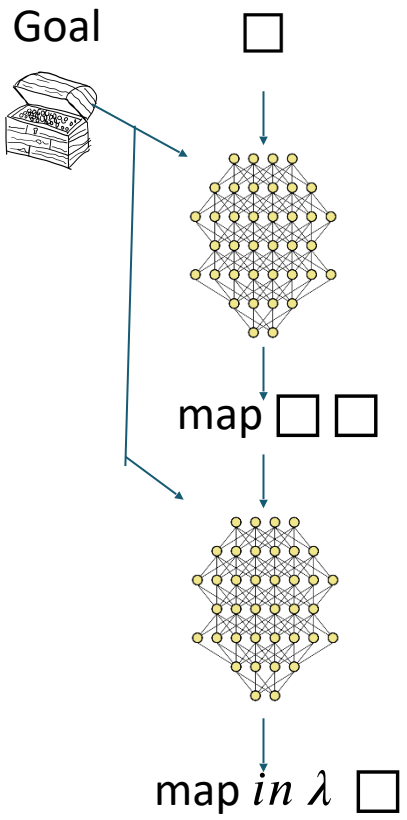
Relaxation



Component Discovery



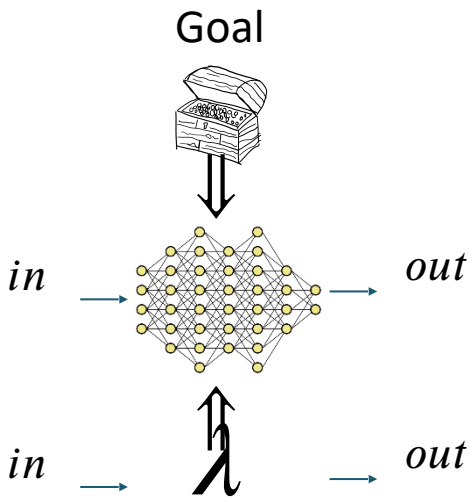
Neural Guided Search



Leverage the ability of NN to learn complex conditional distributions

Network guides the search for programs that satisfy the goals

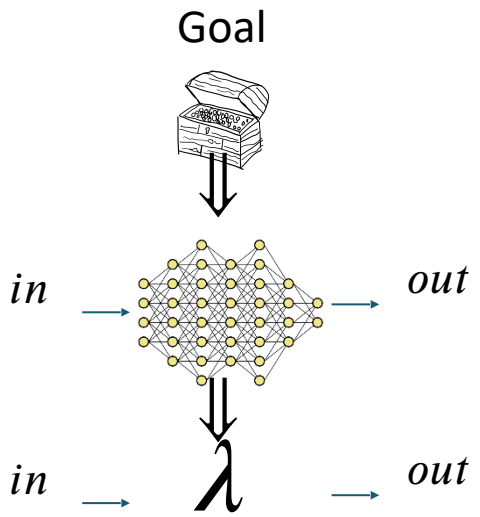
Symbolic Guided Deep Learning



Symbolic knowledge can be used to guide the training of neural networks

- When you want a network that is consistent with prior knowledge
- When you want to improve data efficiency and better generalization

Distillation

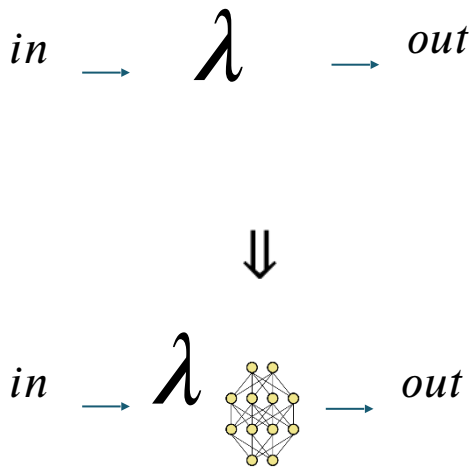


Use the neural network as a starting point for program synthesis

Replace neural components with symbolic ones

- To improve interpretability and analyzability
- To better generalize out of distribution
- To ensure more predictable behavior

Relaxation



Replace symbolic components with neural proxies

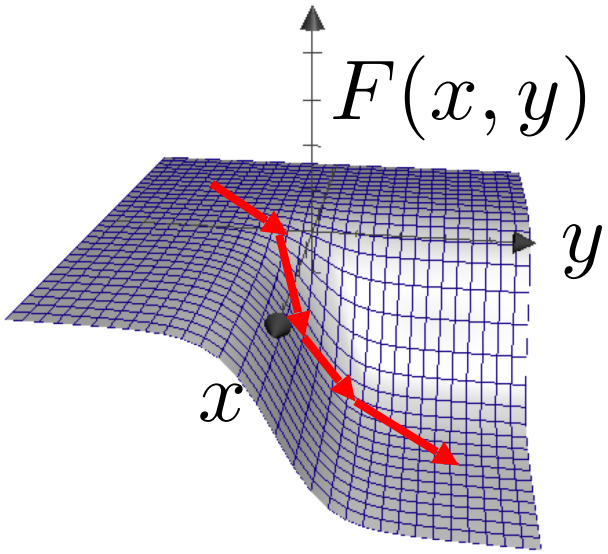
- Can help leverage the information in the symbolic component into a larger DL pipeline
- Can help guide the search for symbolic components

Relaxation:
Supporting numerical optimization over discrete programs

Numerical Optimization

Goal: Find a (hopefully global) minima of a function

- Sample the function on a small set of values
- Based on the result sample in a new set of locations



- Gradient descent
- Newton iteration

…

Numerical Optimization for Synthesis

Many synthesis problems are best framed as optimization problems

- Especially true for problems involving hybrid systems or probabilities

The Parameter Synthesis Problem

```
tOff := ??;    tOn := ??;  h := ??
forever{
    temp := readTemp();
    if (isOn() && temp > tOff)
          switchHeaterOff();
   elseif ( !isOn() && temp < tOn)
         switchHeaterOn(h);
}
```





Warming:

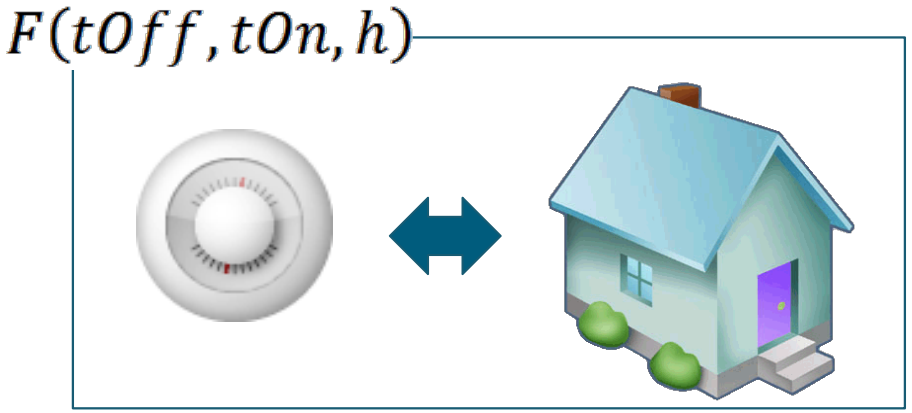
$$\frac{d}{dt}temp = -k \cdot temp + h$$

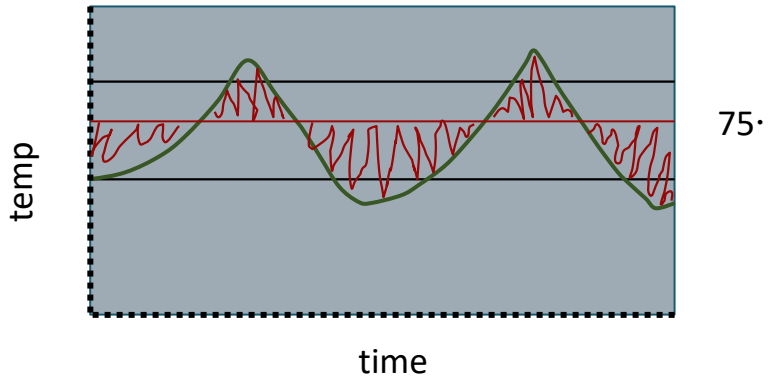
Cooling:

$$\frac{d}{dt}temp = -k \cdot temp$$



The Parameter Synthesis Problem



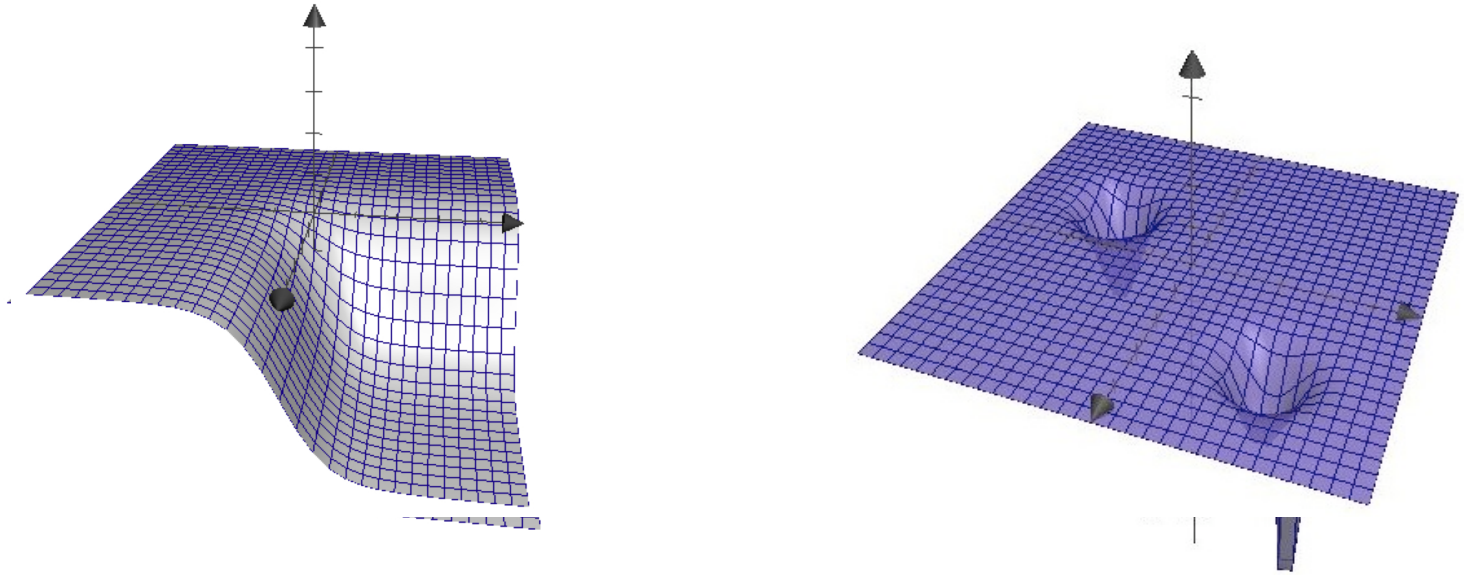


$$F(tOff, tOn, h) = [\, t_0, t_1, \ldots, t_k]$$

$$Err(tOff, tOn, h) := \sum (t_i - 75)^2$$

Can we find values of (tOff, tOn, h) to minimize Err?

The challenge of optimizing programs

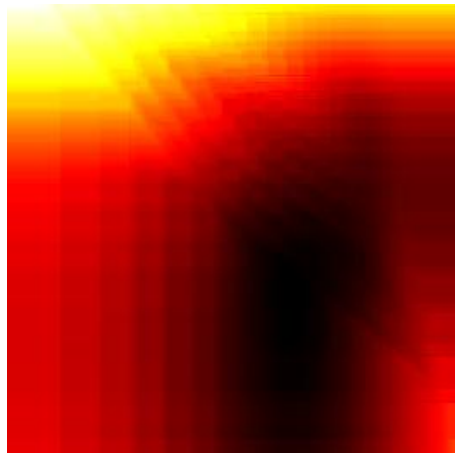


Plateaus and discontinuities thwart search

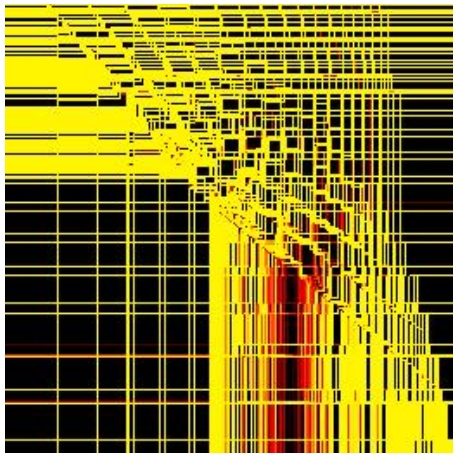
Branches introduce discontinuities

For the thermostat example

$Err(tOn, tOff, 150)$

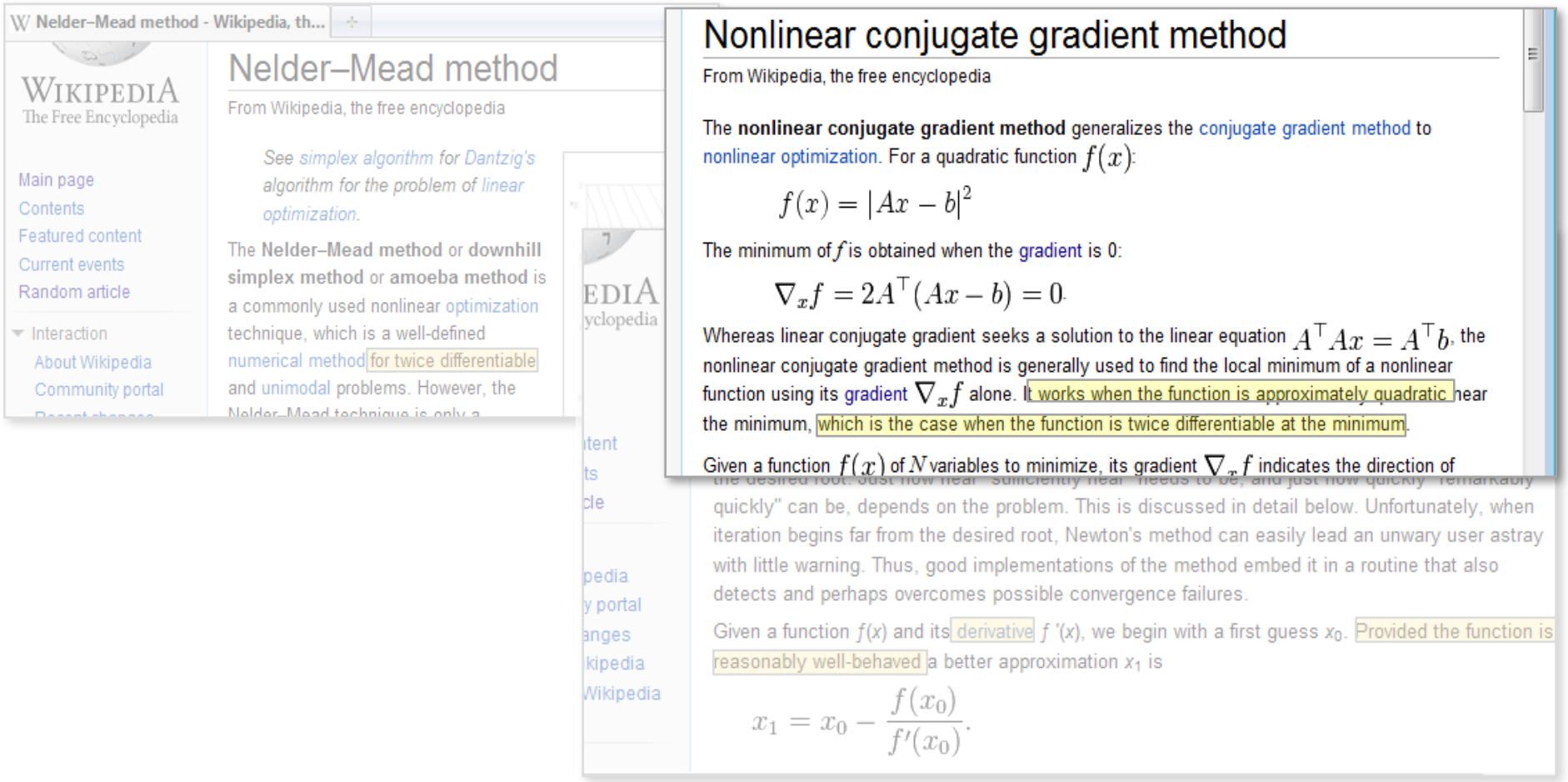


$|\vec{\nabla} Err(tOn, tOff, 150)|$

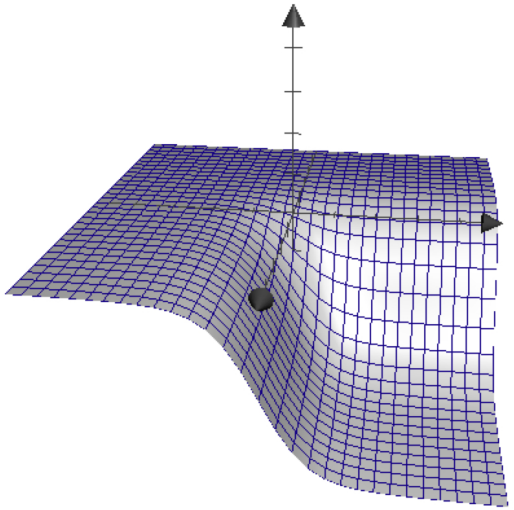


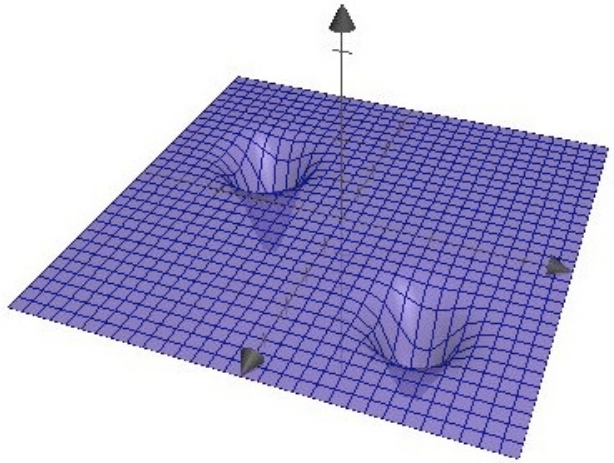
Minimization by numerical Search

Discontinuities affect even the best methods



Key Idea: Smooth the Function

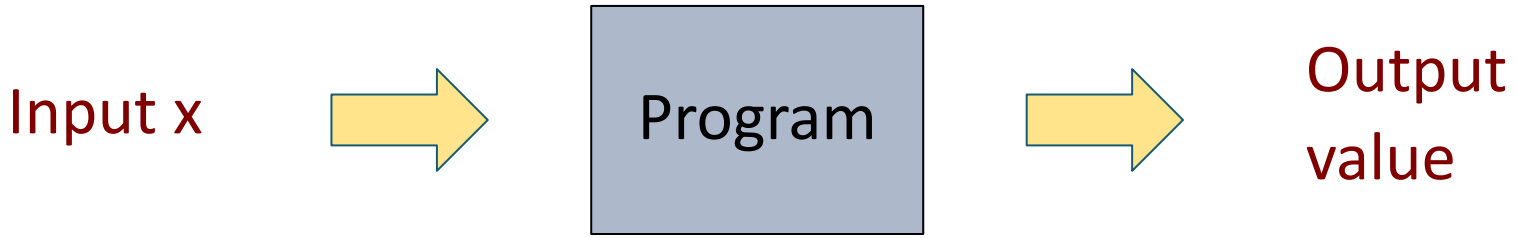




Approximate the problem by a smooth function

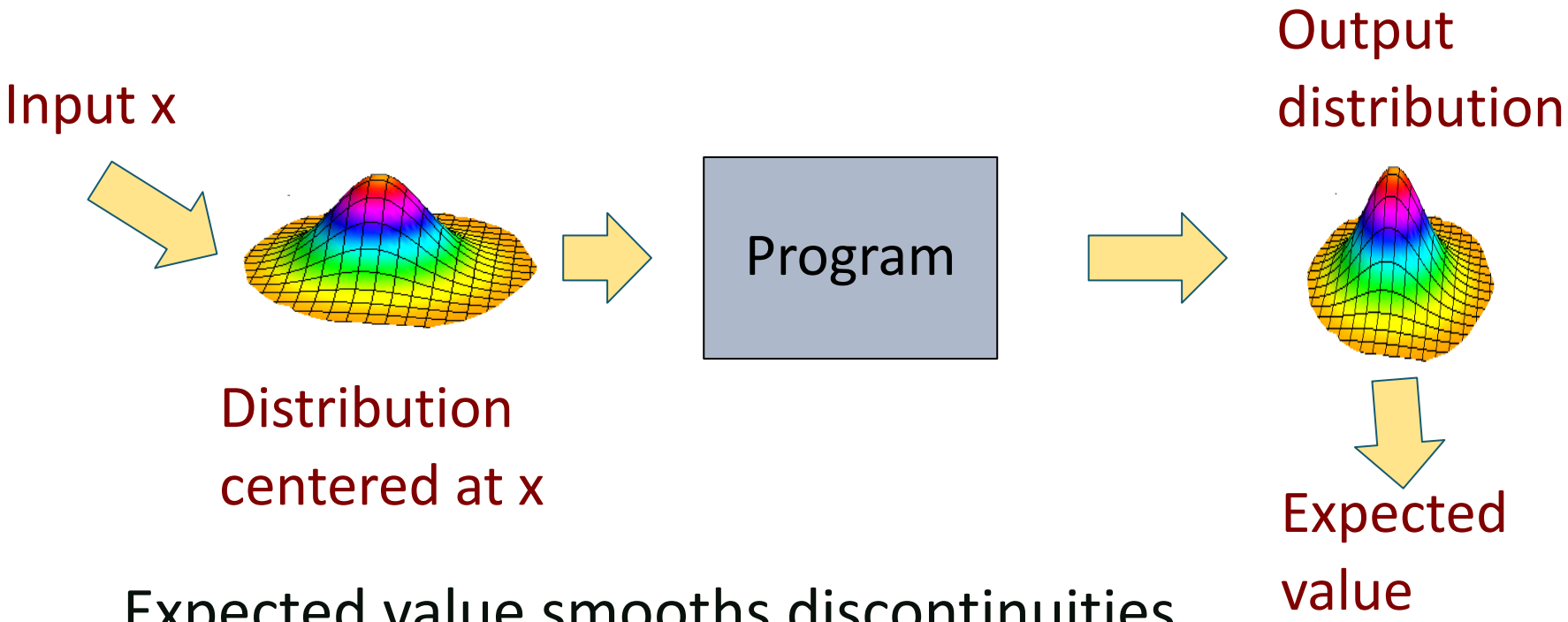
Example: Gaussian Smoothing

Execute on a distribution instead of a value



Example: Gaussian Smoothing

Execute on a distribution instead of a value

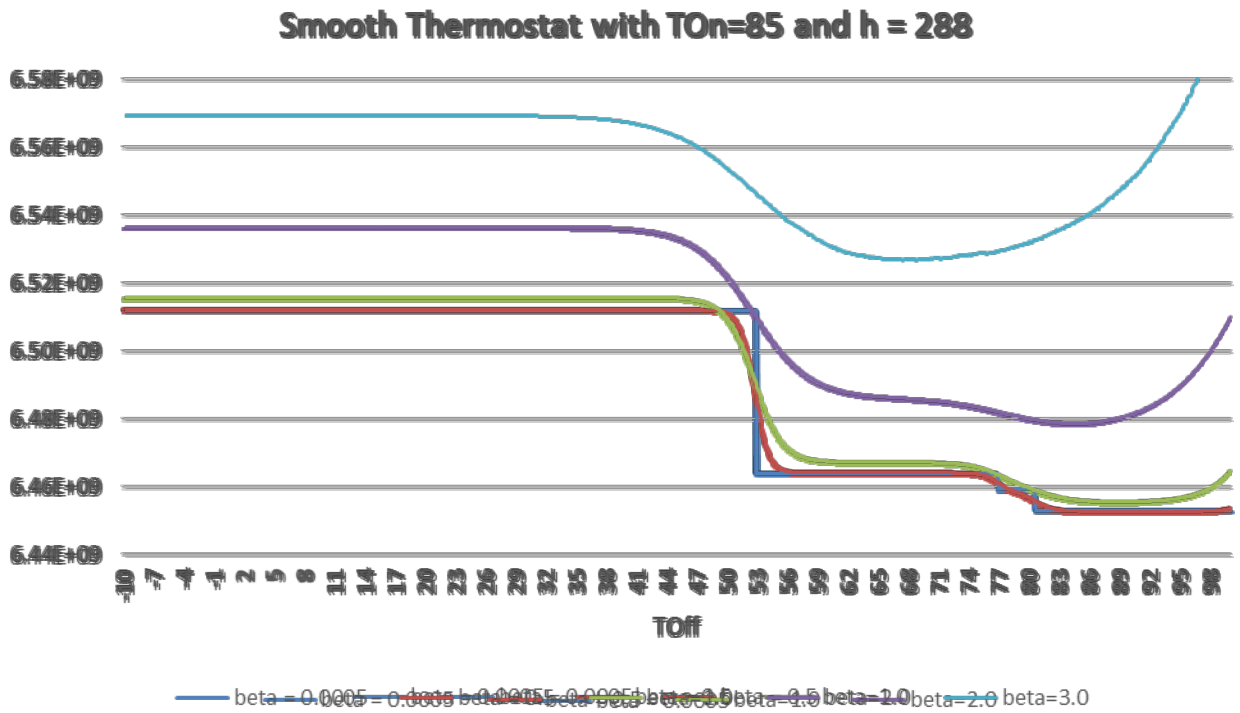


Expected value smooths discontinuities

Parameterized Smoothing

Parameterized by a value β

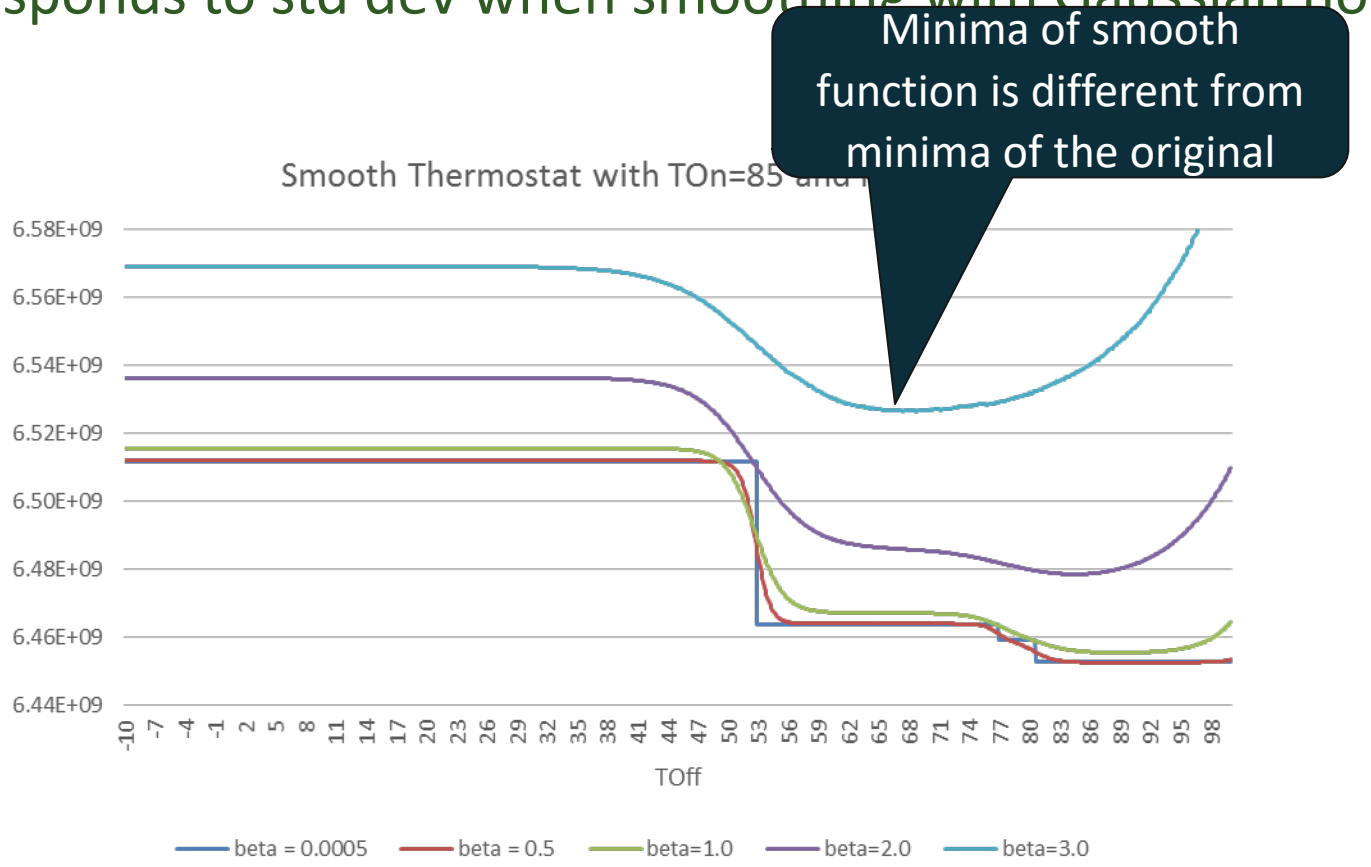
- Corresponds to std dev when smoothing with Gaussian noise



Parameterized Smoothing

$F_{smooth}(p,\ \beta_i)$ parameterized by a value β

- Corresponds to std dev when smoothing with Gaussian noise



Smoothed Numerical Search

$p_0 = random\ starting\ point$
$\beta_0 = initial\ beta$
$While\ \beta_i > \epsilon$
$p_{i+1} = optimize\ F_{smooth}(p,\ \beta_i)\ starting\ at\ p_i$
$\beta_{i+1} = \beta_i * q$
$return\ p_i$

$0 \leq q < 1$

Smoothed Numerical Search

Very general procedure

Steps:

- Frame your problem as an optimization problem
- Define a parameterized smoothing procedure
- Run the algorithm

Smoothing doesn't have to be perfect

- But it should approximate the desired function in the limit as $\beta \to 0$

Euler:
SNS for programs

SNS in Euler

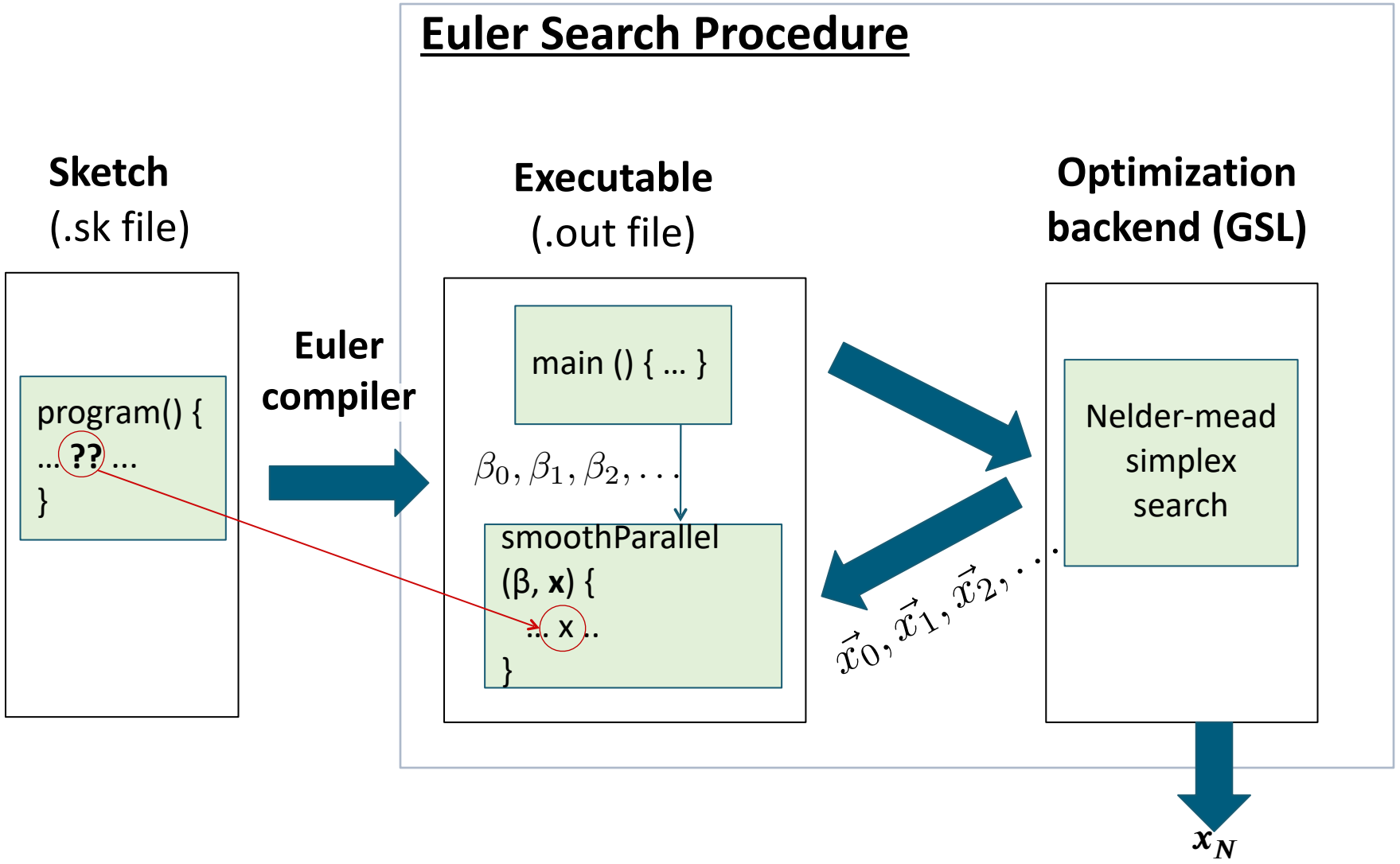
Use Symbolic Gaussian Smoothing

- β is simply the standard deviation

Quantifier alternation is handled through fixed test harnesses

- not ideal,
- but works well in practice

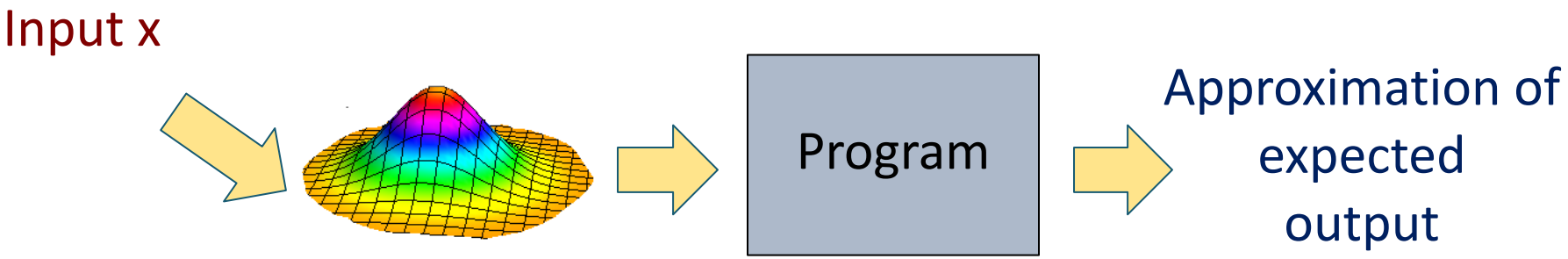
Smoothed Numerical Search in Euler



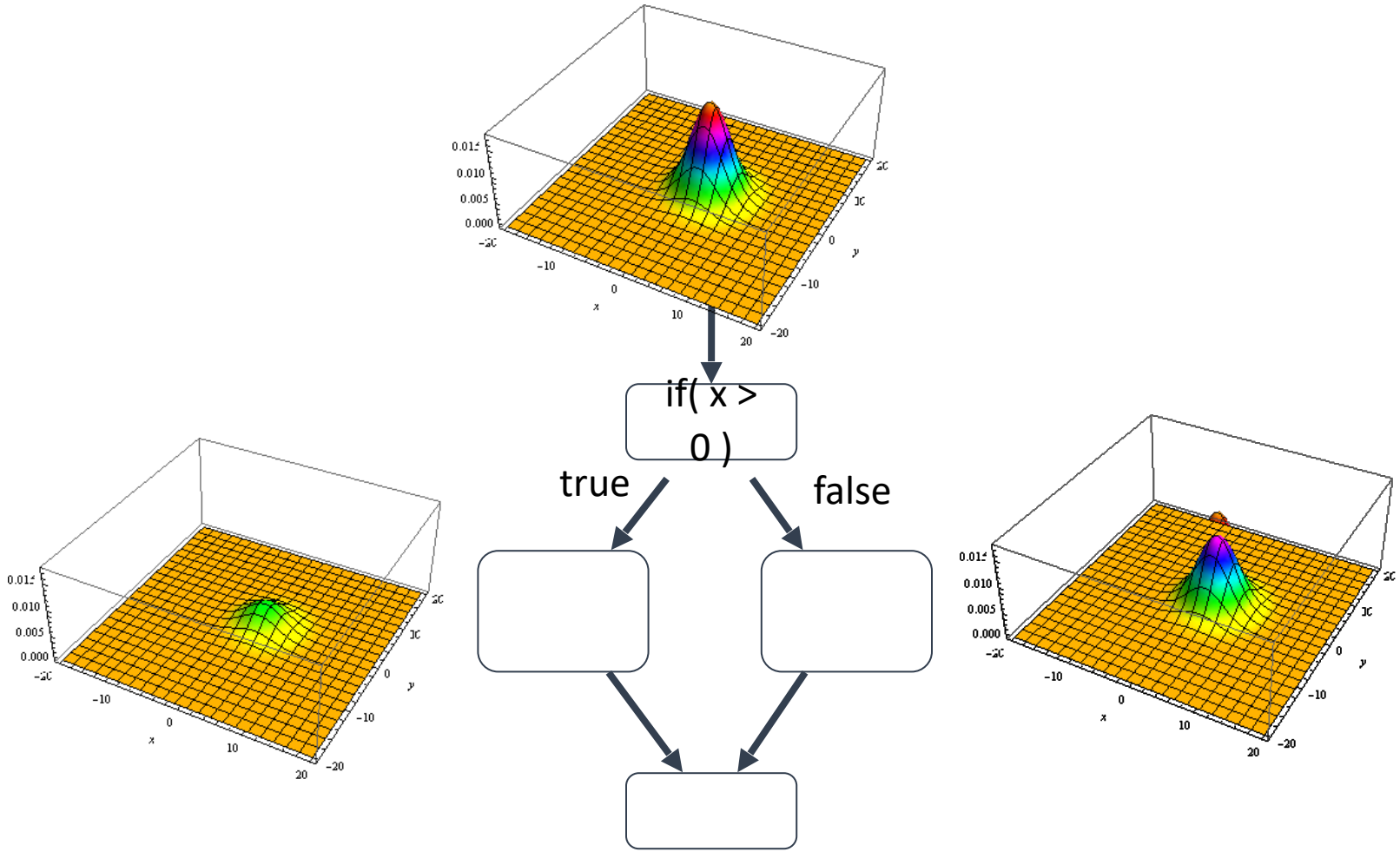
Smooth Interpretation

How do we execute a program on a distribution of inputs?

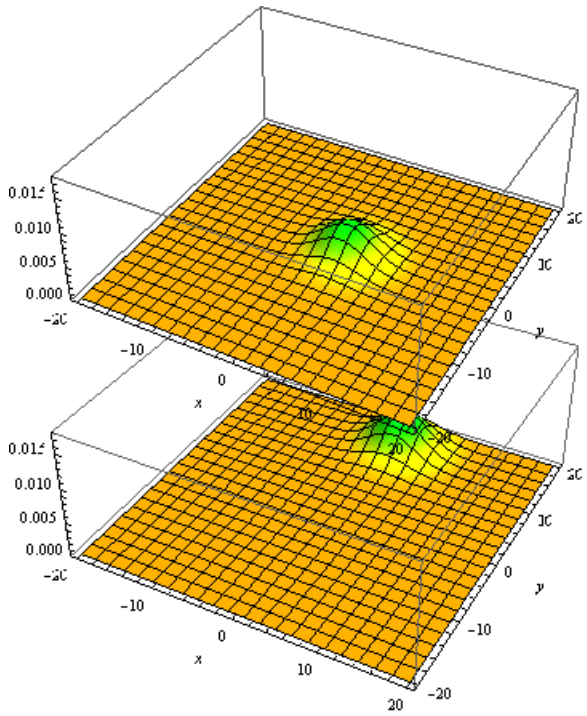
- Not by sampling
  - we would miss essential features of the search landscape.
- Symbolic execution:
  - propagate symbolic representations of distributions.

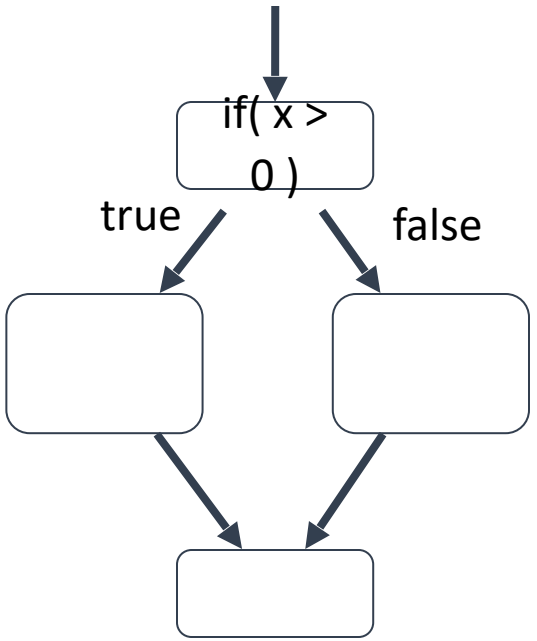


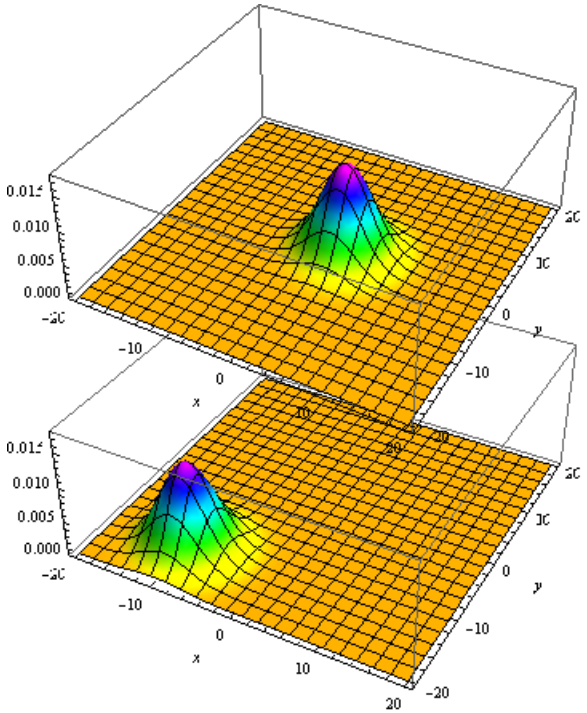
Smooth Interpretation: Branch

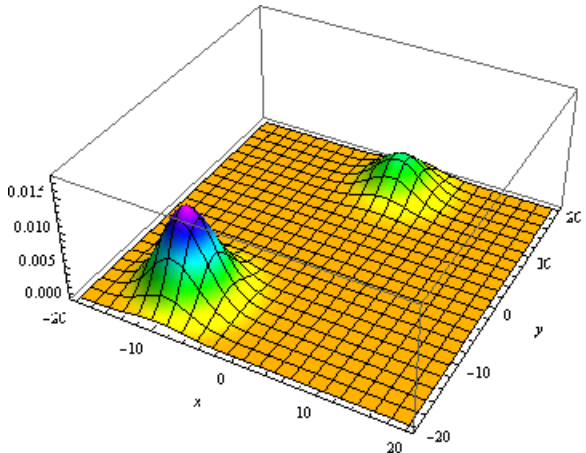


Smooth Interpretation: Join

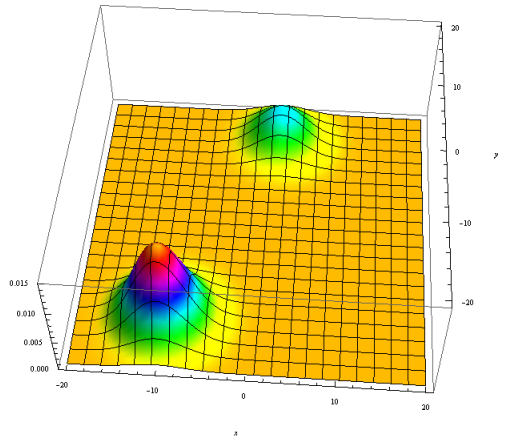


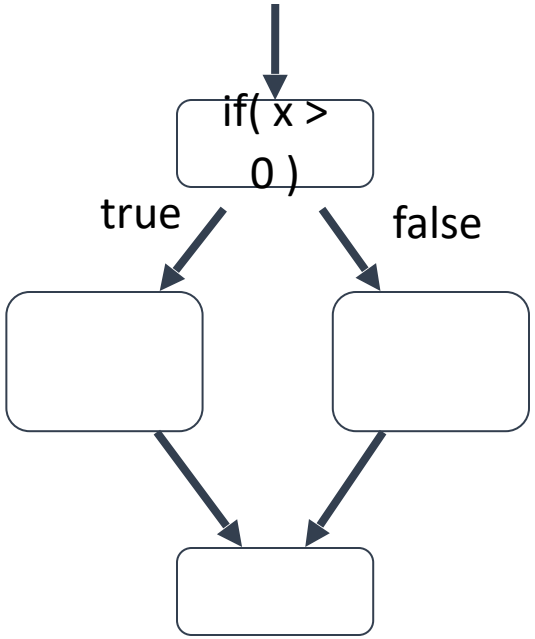


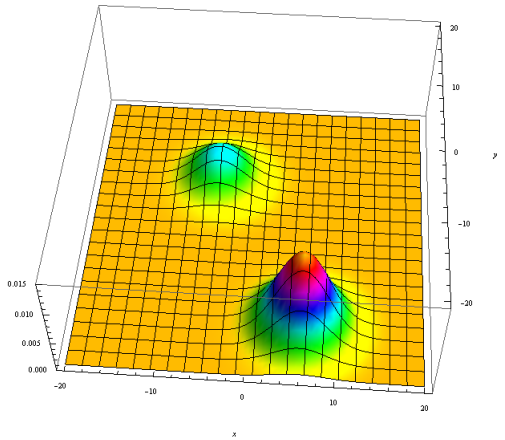


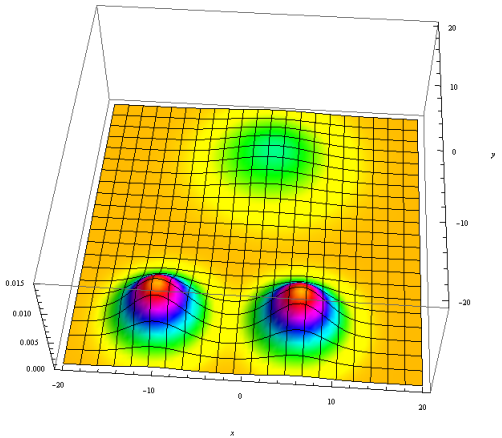


Smooth Interpretation: Join

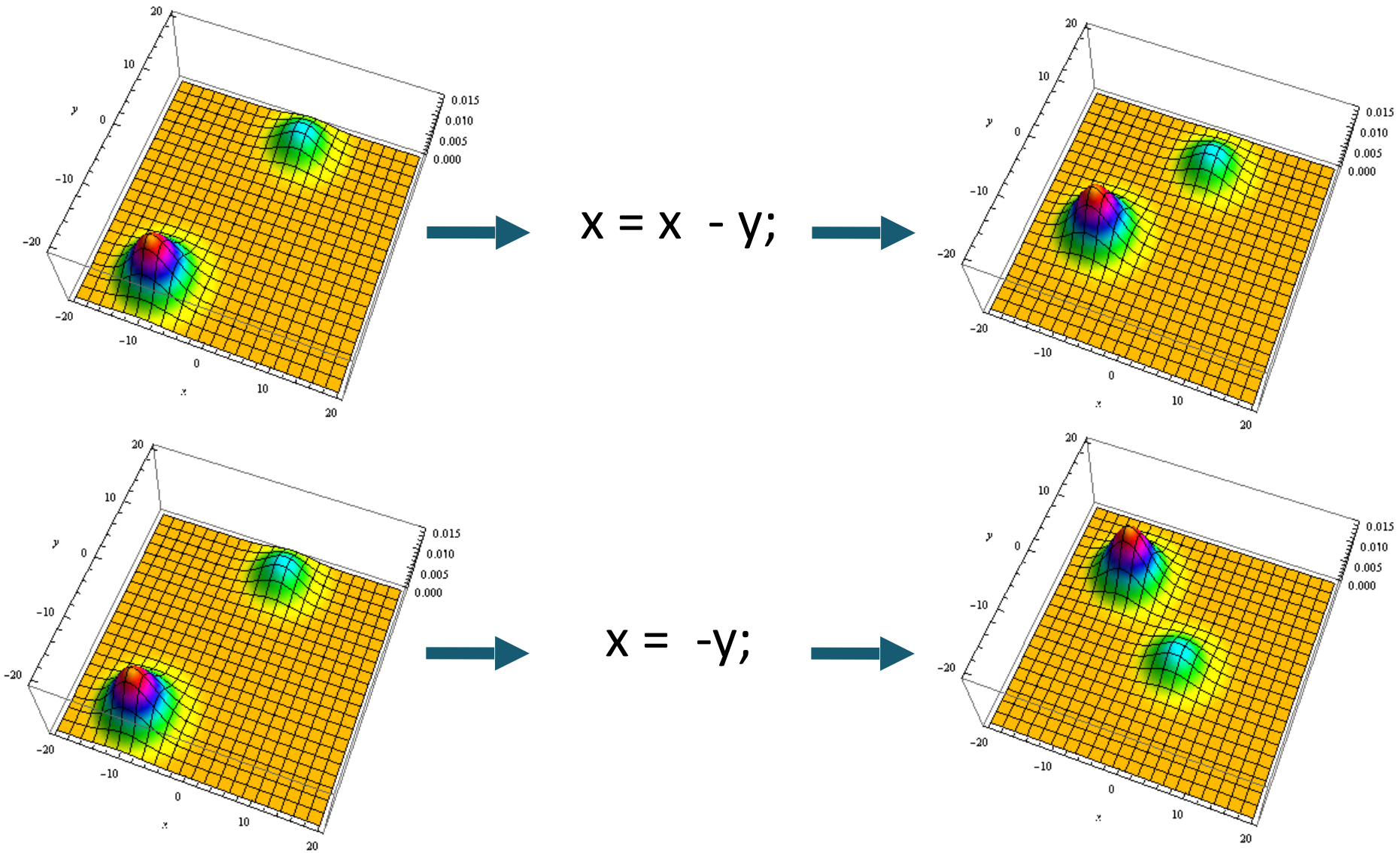








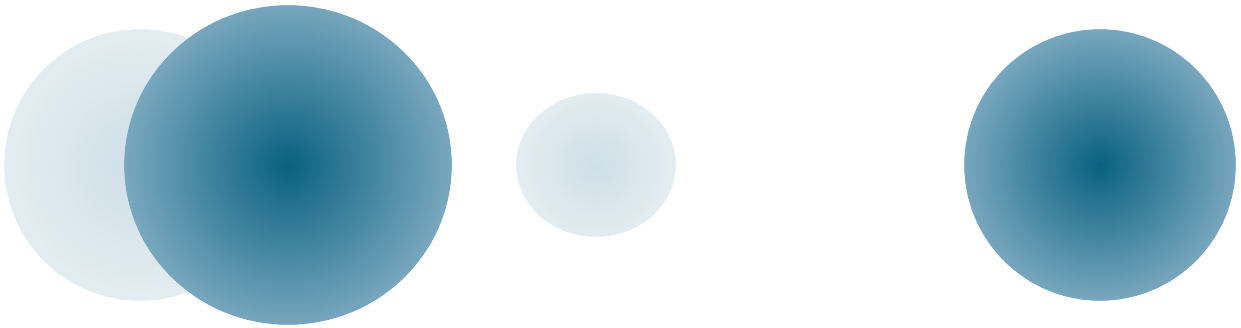
Smooth Interpretation: Assignment



Approximations

Trick Question:

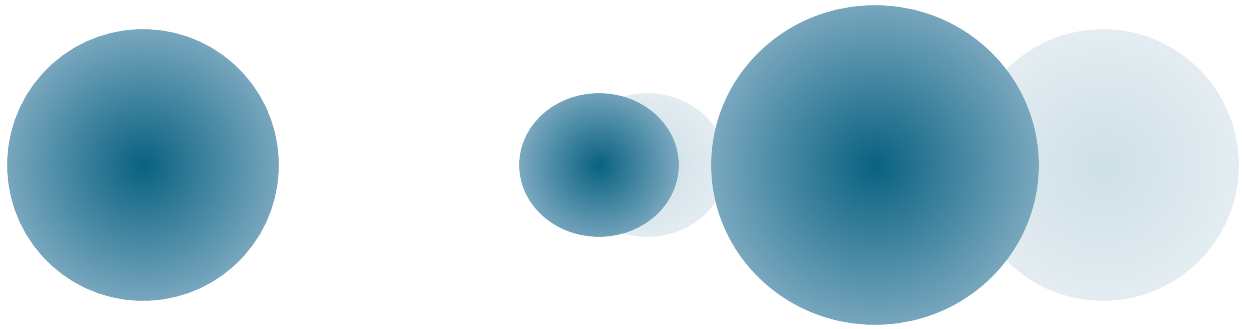
- Which of the previous approximations is problematic?



Approximations

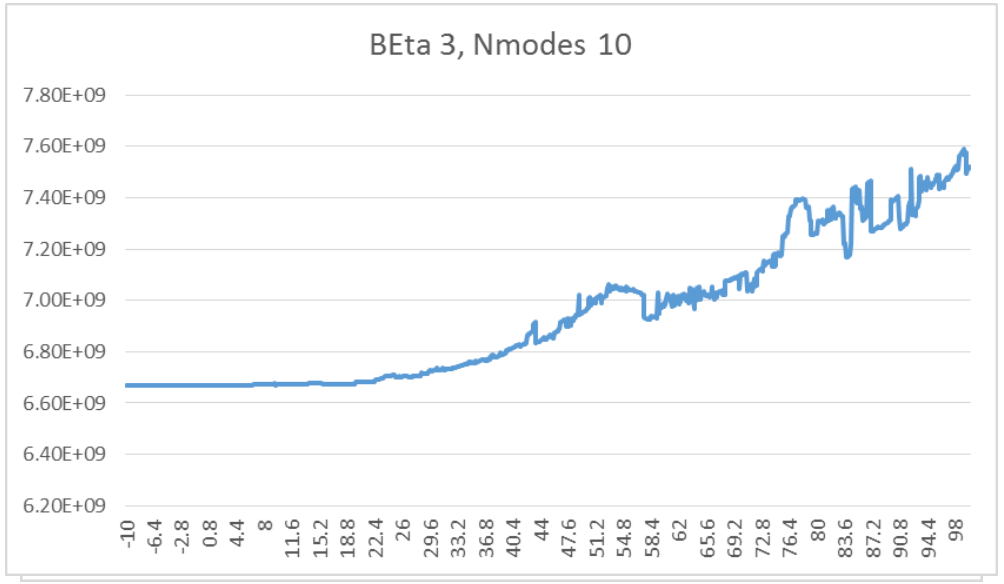
Trick Question:

- Which of the previous approximations is problematic?



Merging can introduce discontinuities!

Excessive merging can cause problems



Examples

Parameter synthesis

t_1 := **??**; t_2 := **??**; A_1 := **??**;

A_2 := **??**;

repeat ~~**(every ΔT)**~~ {

if (stage = BACK && time > t_1)

stage = INTURN;

if (stage = INTURN) {

if (time > t_2) **then** stage = OUTTURN;

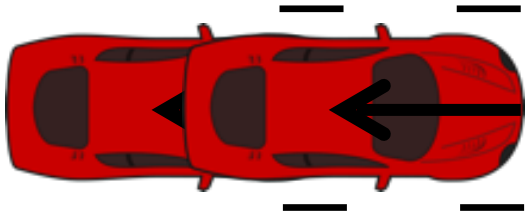
angle = angle – A_1; }

if (stage = OUTTURN)

angle = angle + A_2;

moveCar();

}







Parameter synthesis

t_1 := **??**; t_2 := **??**; A_1 := **??**;

A_2 := **??**;

repeat (every ΔT) {

if (stage = BACK && time > t_1)

stage = INTURN;

if (stage = INTURN) {

if (time > t_2) **then** stage = OUTTURN;

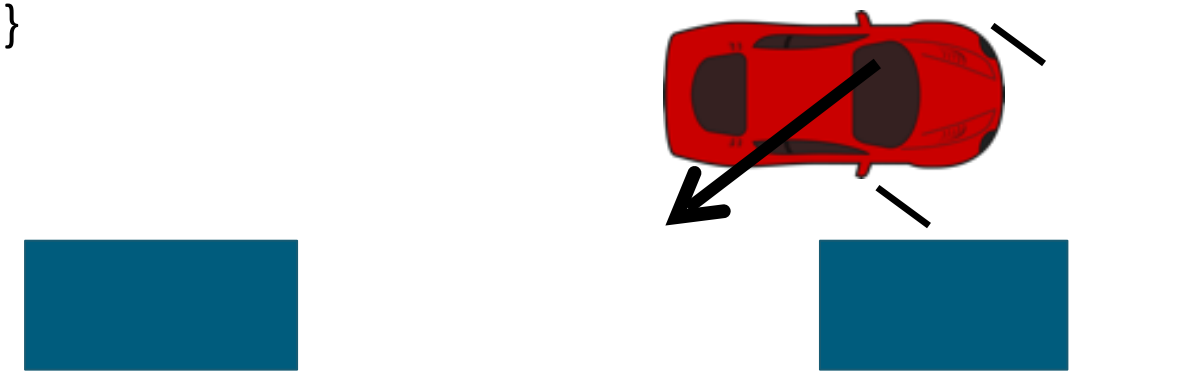
angle = angle – A_1; }

if (stage = OUTTURN)

angle = angle + A_2;

moveCar();

}



Parameter synthesis

t_1 := **??**; t_2 := **??**; A_1 := **??**;

A_2 := **??**;

repeat (every ΔT) {

if (stage = BACK && time > t_1)

stage = INTURN;

if (stage = INTURN) {

if (time > t_2) **then** stage = OUTTURN;

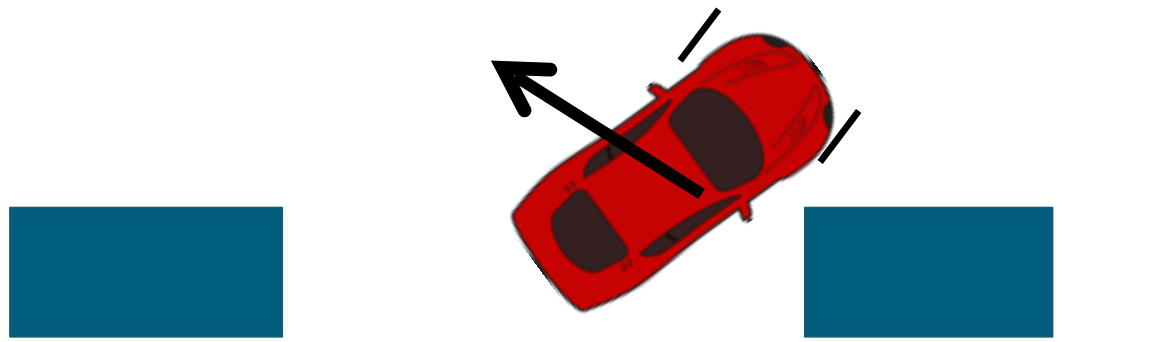
angle = angle – A_1; }

if (stage = OUTTURN)

angle = angle + A_2;

moveCar();

}



Application : Parameter synthesis

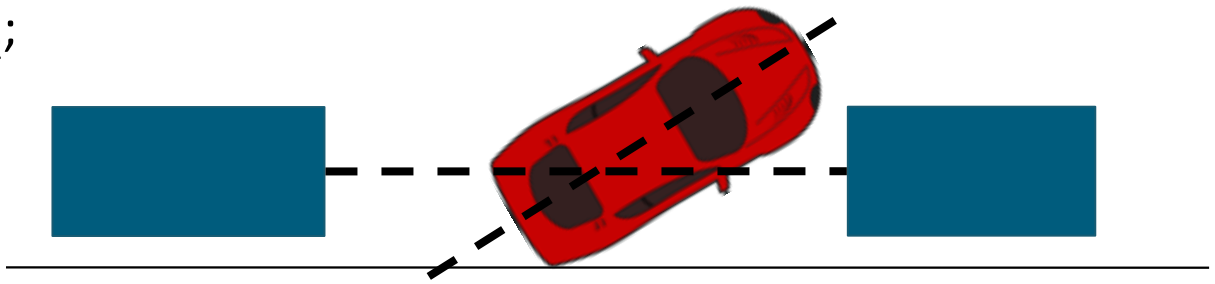
t_1 := **??**; t_2 := **??**; A_1 := **??**;

A_2 := **??**;

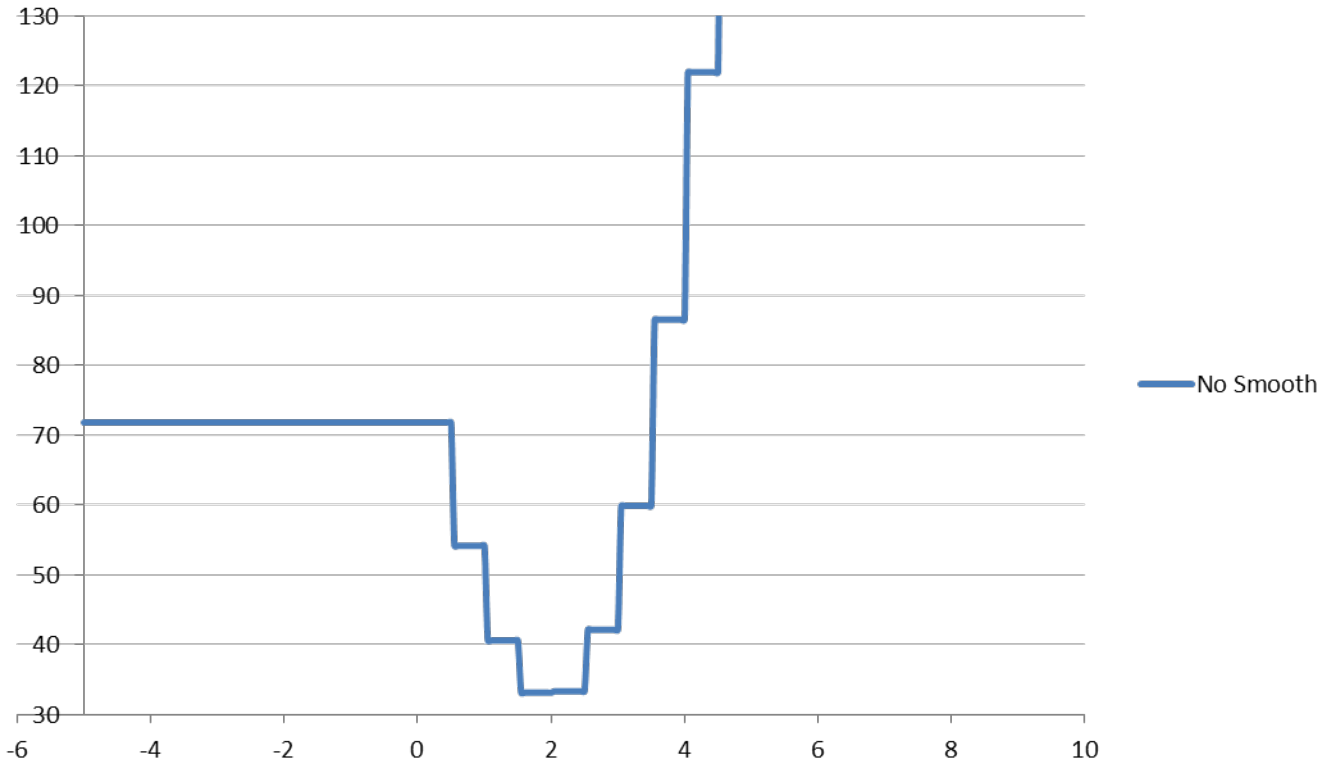
```
repeat (every ΔT) {
    if (stage = BACK && time > t1)
        stage = INTURN;
    if (stage = INTURN) {
        if (time > t2) then stage = OUTTURN;
        angle = angle – A1; }
    if (stage = OUTTURN)
        angle = angle + A2;
  moveCar();
}
```

Error = distance between ideal and actual final position

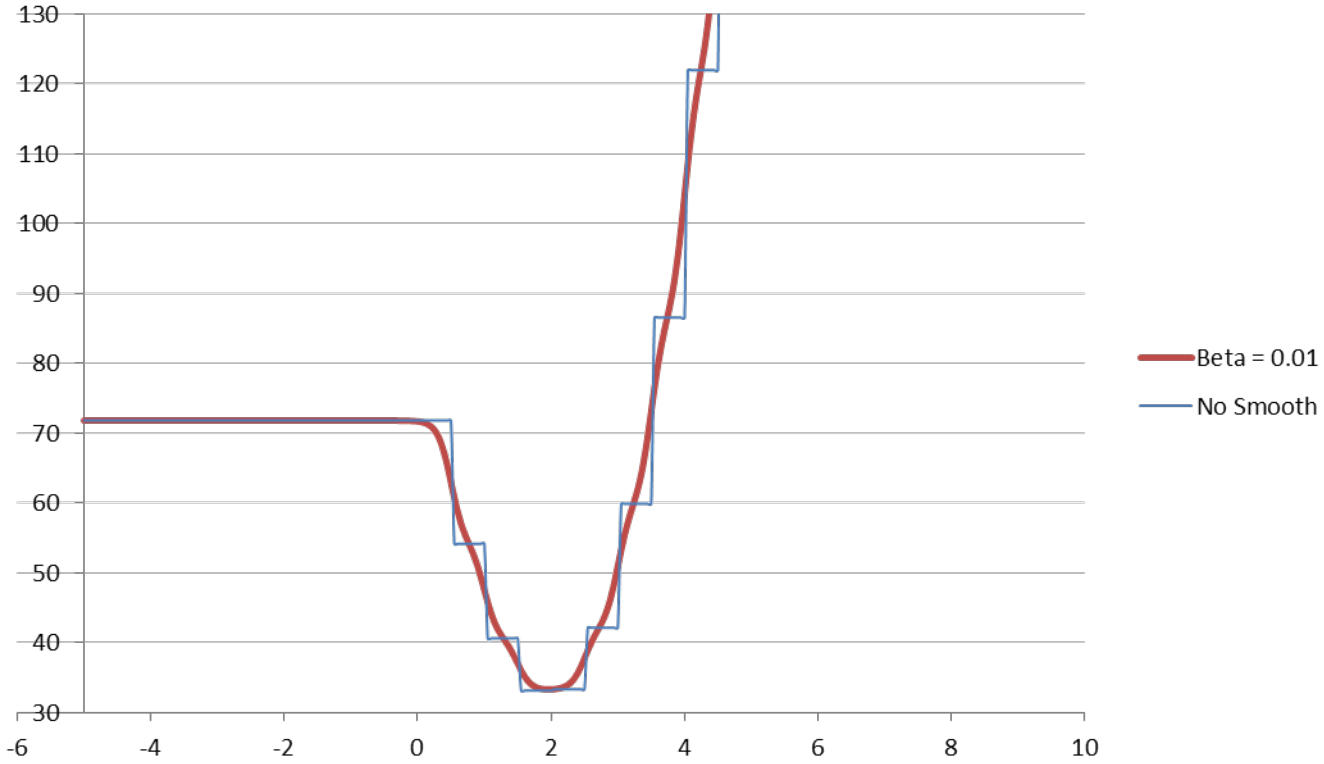
Goal: minimize Error.



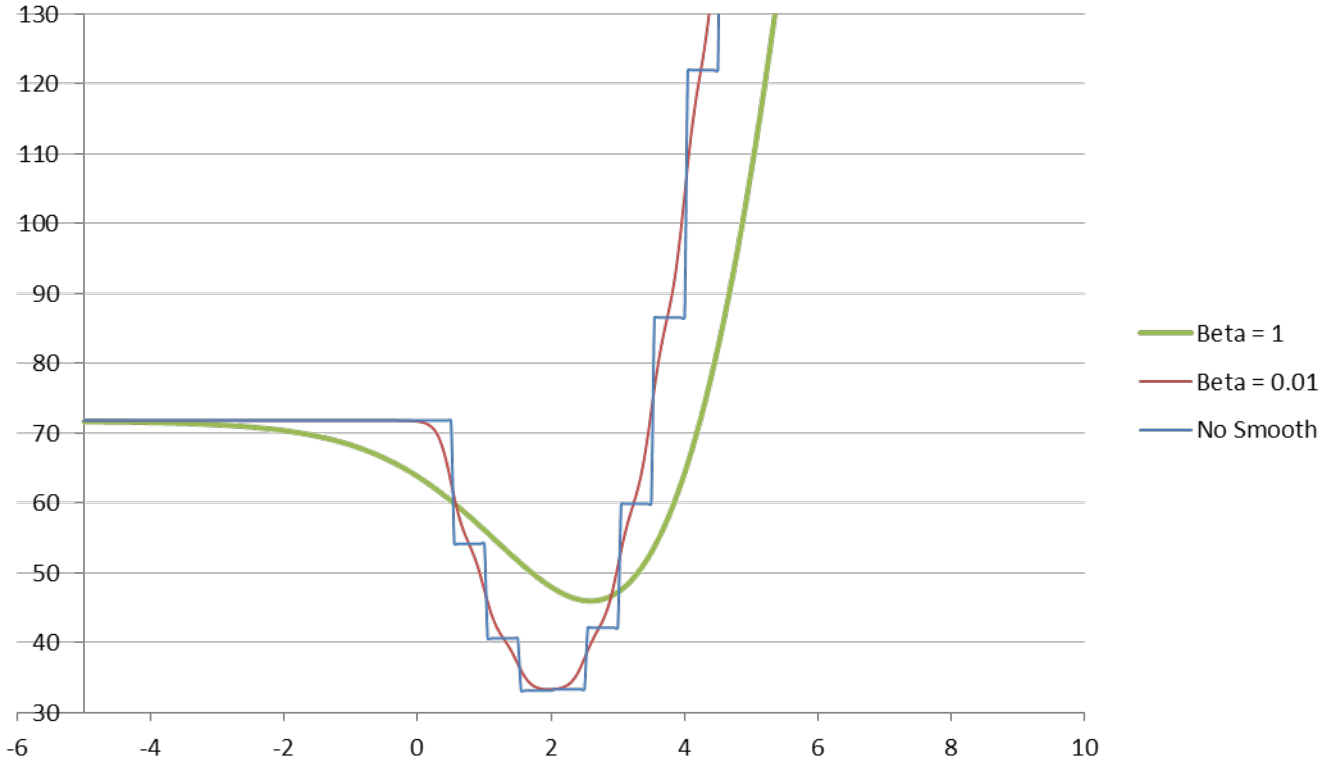
Slice of Error function WRT T2



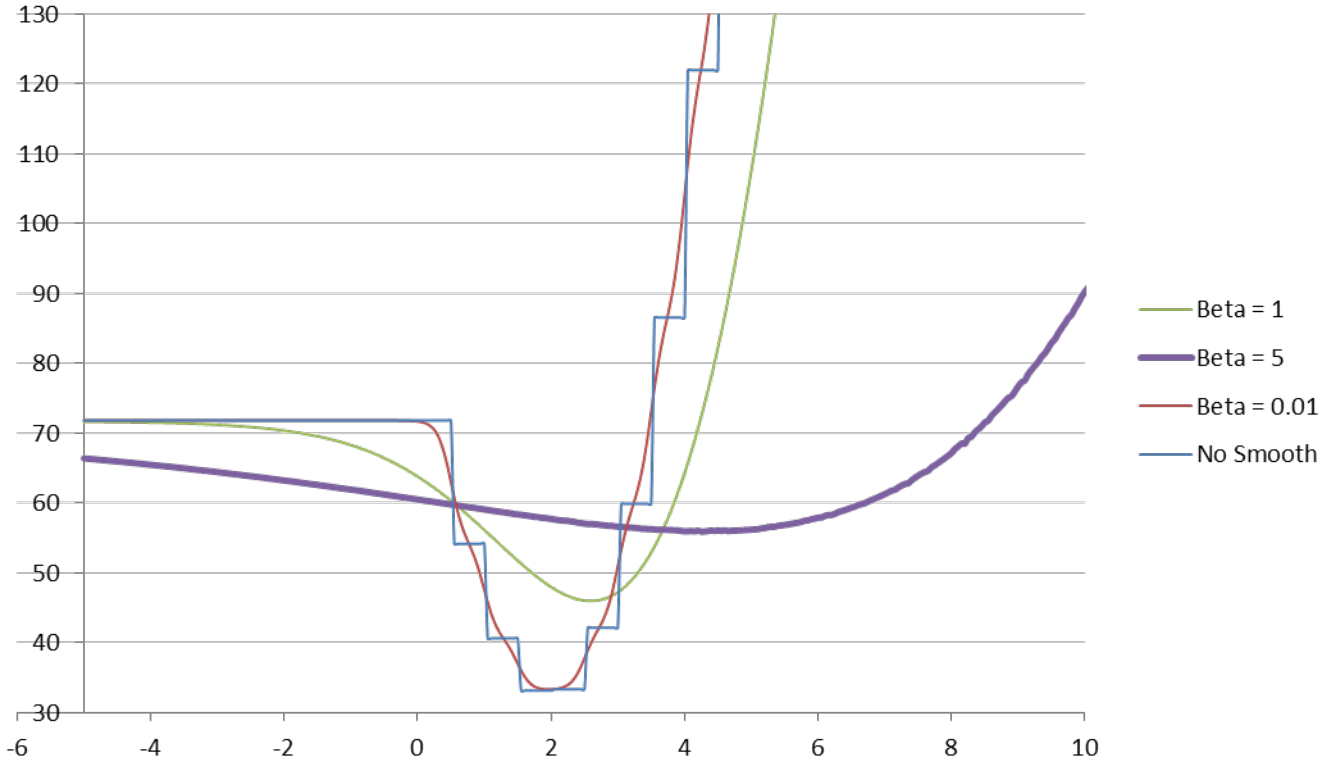
Slice of Error function WRT T2



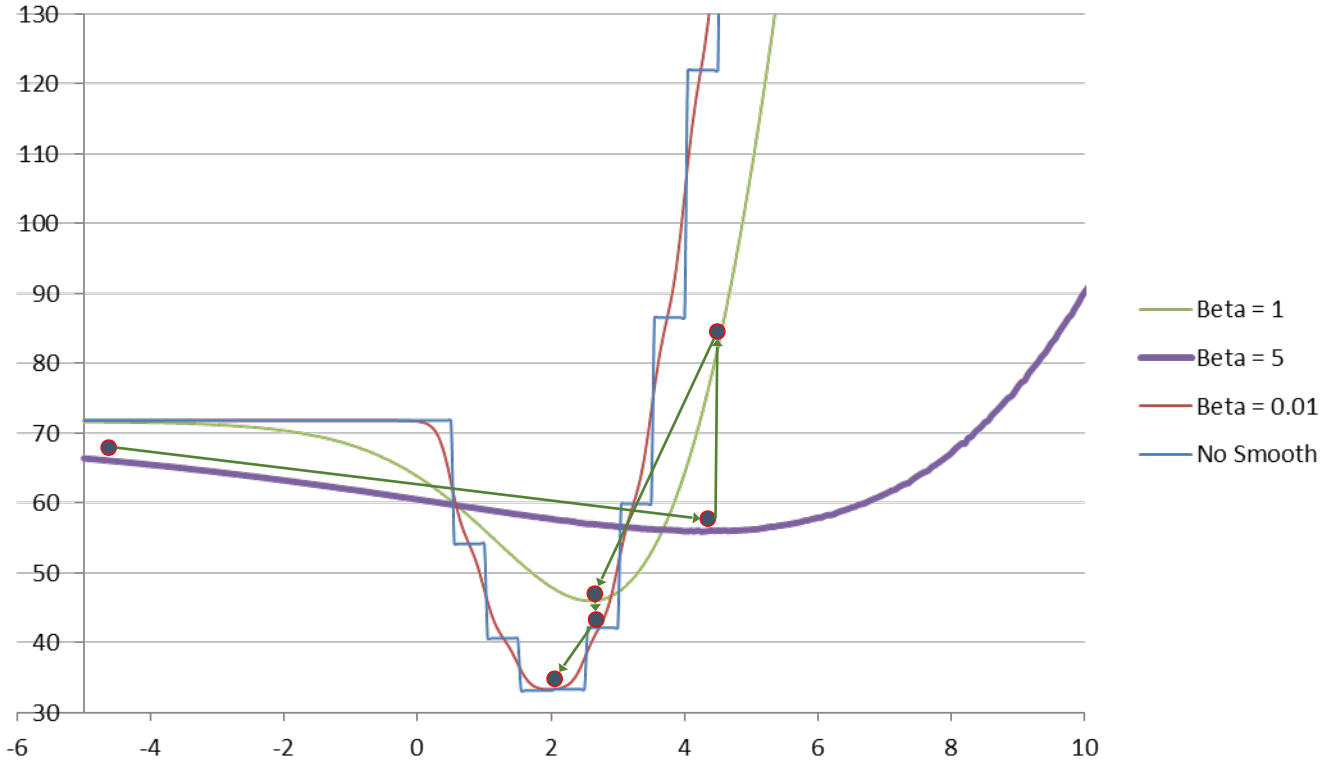
Slice of Error function WRT T2



Slice of Error function WRT T2



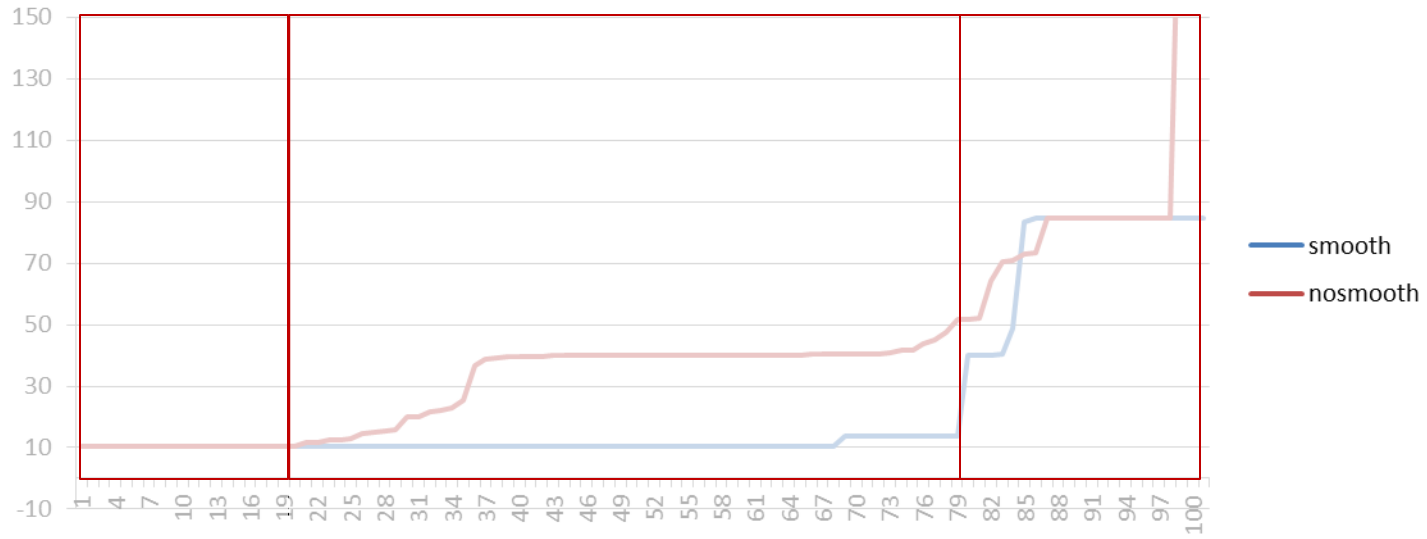
Numerical Search with Smoothing



Errors from different starting points

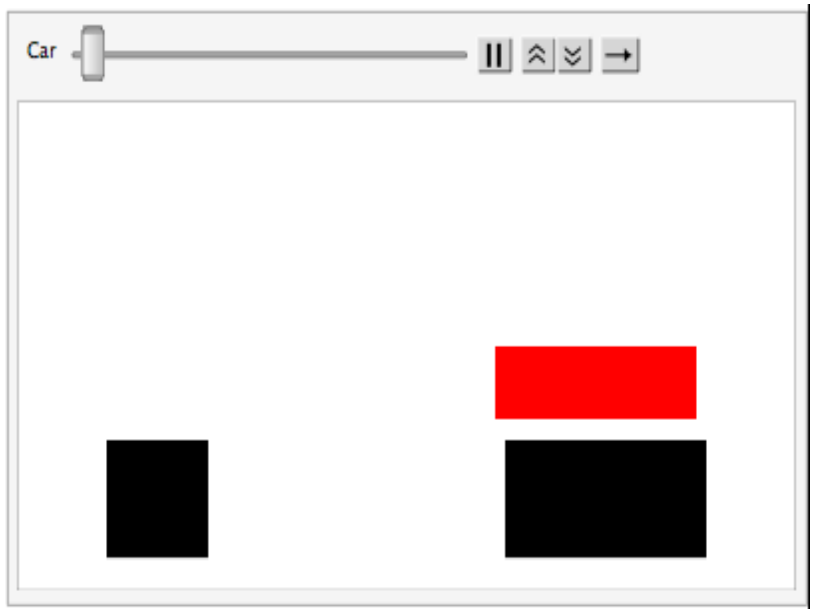
Both are quite good

Both are quite bad

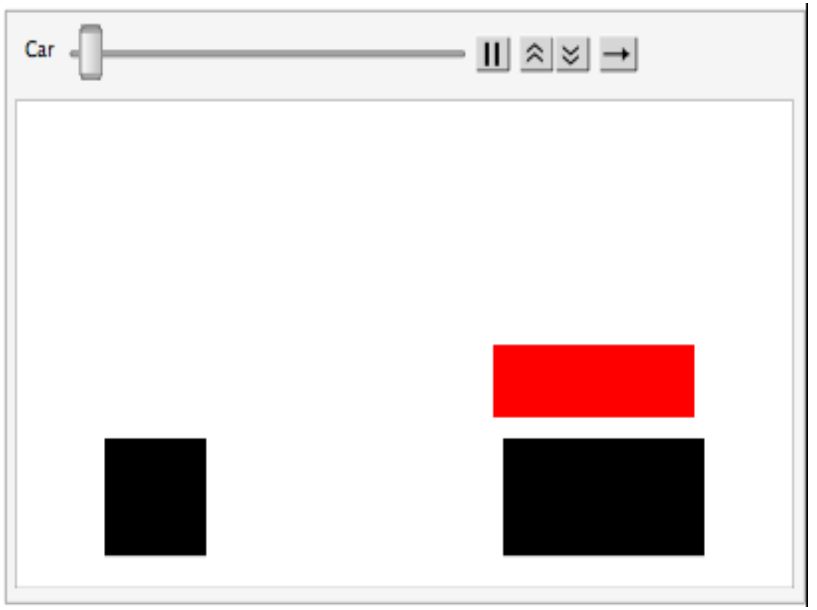


Result of smoothing is much better

What the difference means



How an error of 40 look like

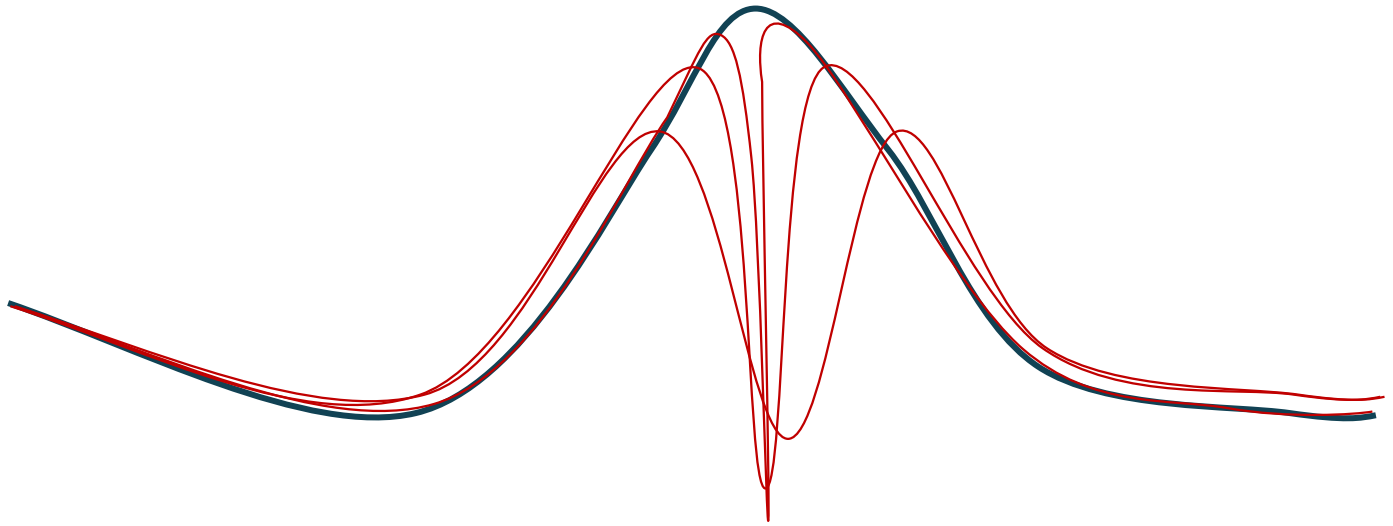


How an error of 10 looks like

Limitations

Smooth function converges to original function according to L2 norm.

- That’s not enough for optimization.
- The function below converges to the original function, but the local minima is in the worst possible point



Limitations

Programs tend to be difficult to optimize

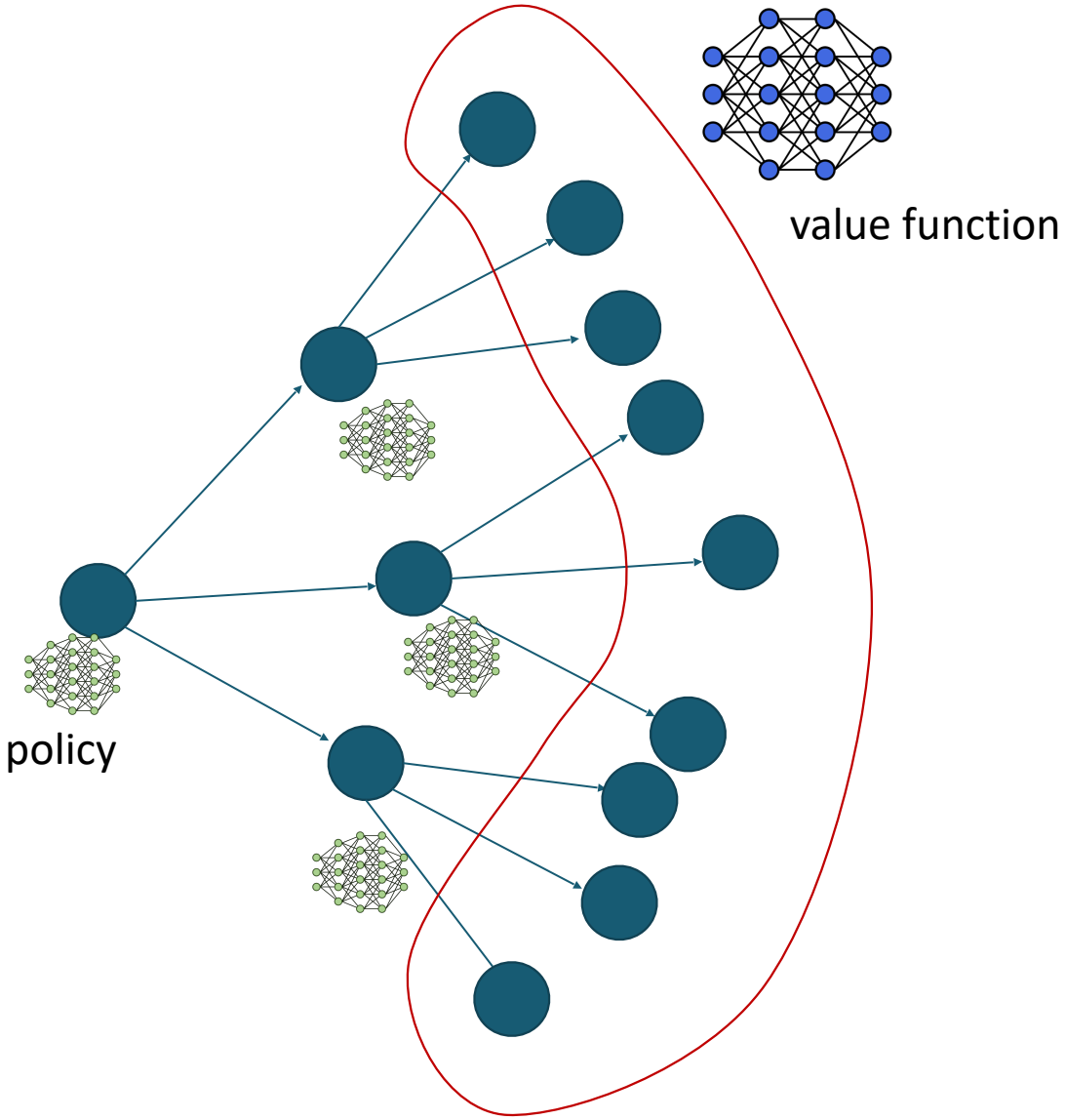
- Even with smoothing, there are too many local minima

Can be useful in combination with other search techniques

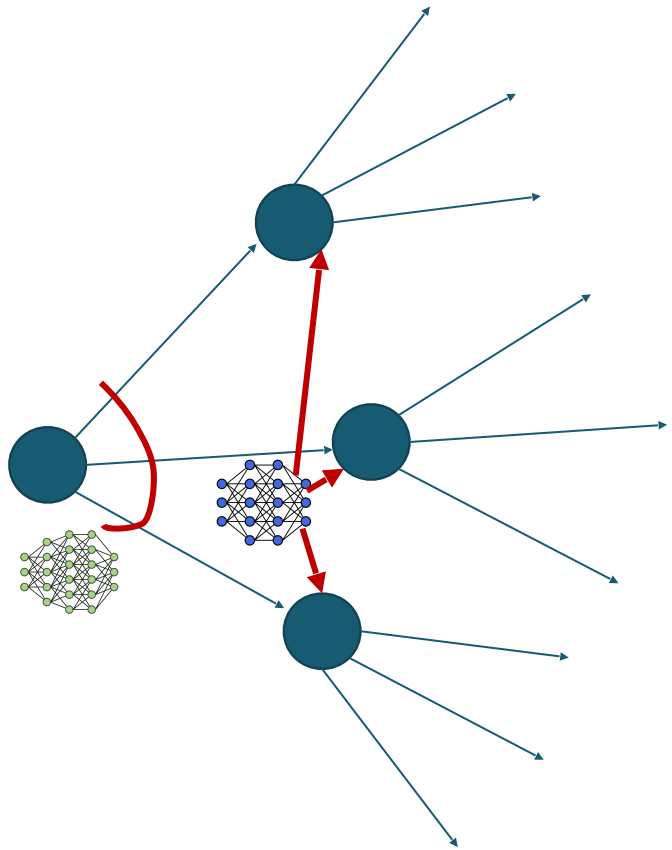
- e.g. Hamiltonian Monte Carlo

Relaxation + Top-Down ≈ Near

Learning to synthesize incrementally 2



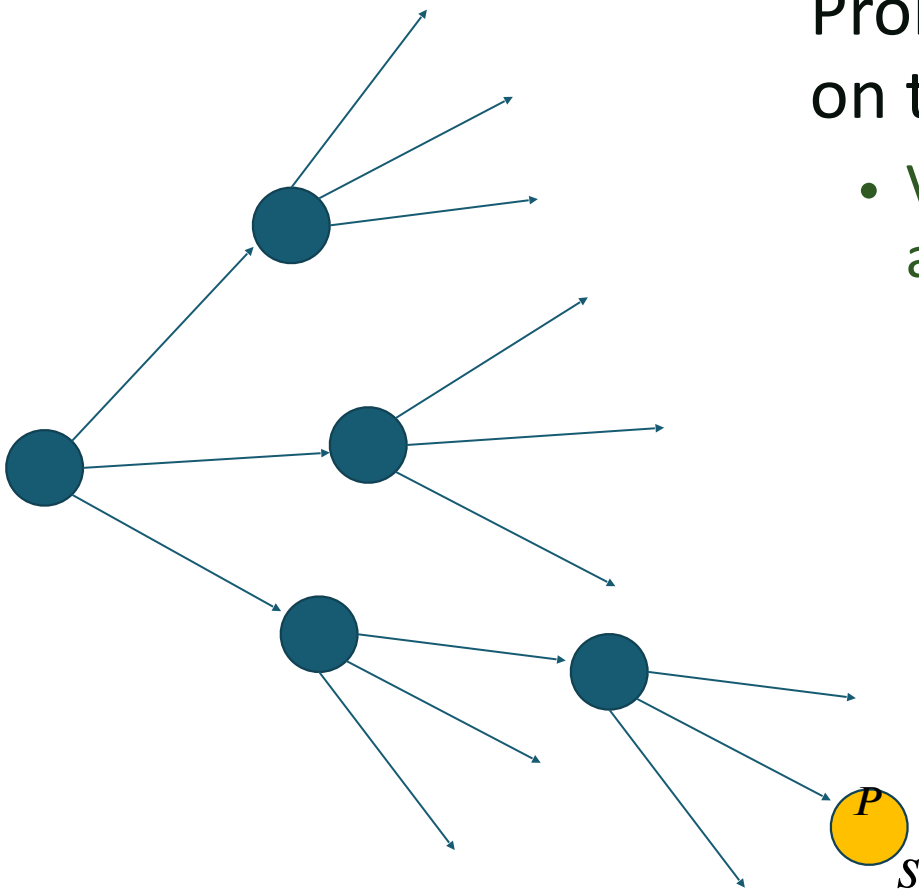
Guiding program search



So far

- Neural network policy to guide search
- Value function scores individual states

Guiding program search

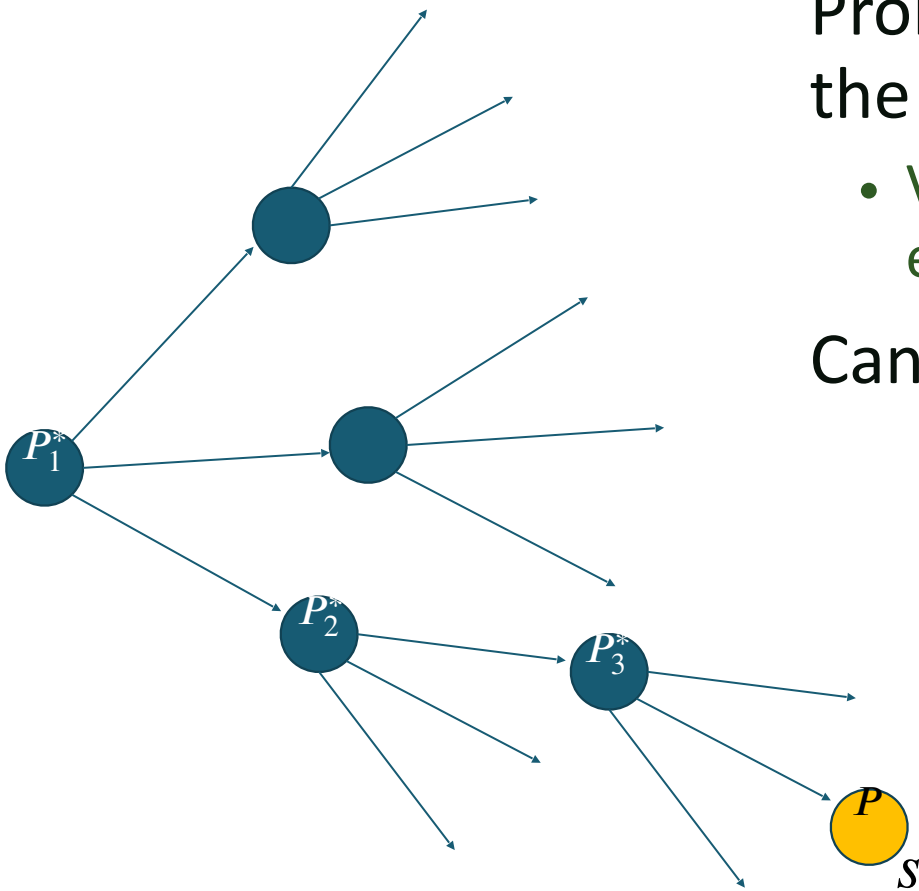


Problem: You only get ground truth on the leaves of the search tree

- Value for an intermediate node is only an estimate

$$s(P) + \min_{\theta} \text{Loss}(\alpha_P, \theta_P)$$

Guiding program search



Problem: You only get ground truth on the leaves of the search tree

- Value for an intermediate node is only an estimate

Can we get a better estimate with DL?

$$s(P) + \min_{\theta} \text{Loss}(\alpha_P, \theta_P)$$

Estimating the "Cost to Go"

P^* = partial program (non-terminal nodes)

$P^*[? \rightarrow e]$ = completions of P^* (reachable terminal nodes)

"Cost to Go"

Heuristic Estimate: $d(P^*) \approx \min_{P \in \mathbb{C}(P^*)} \left[\Delta s(P, P^*) + \min_{\theta} \text{Loss}(\alpha_P, \theta_P) \right]$

Additional Structure Cost

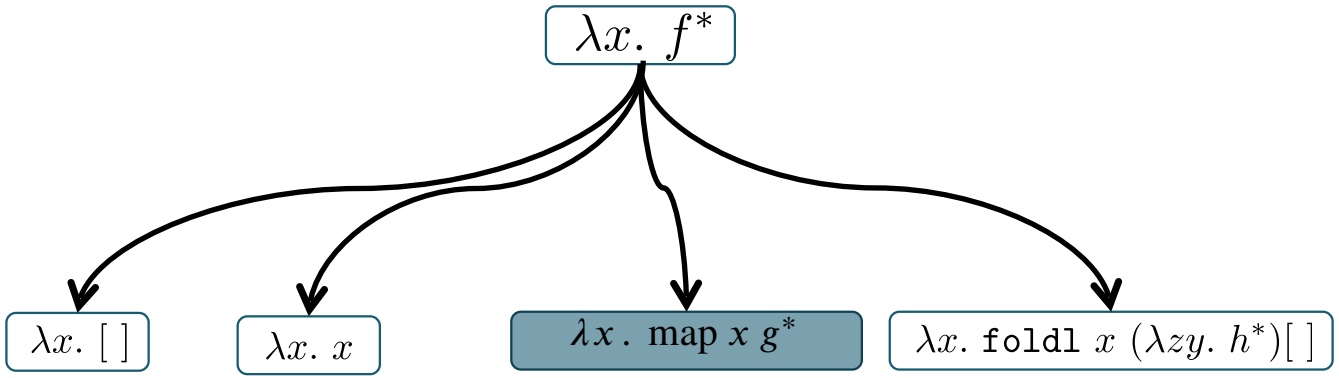
Training Loss

If $d(P^*)$ is a lower bound it becomes an "admissible heuristic"

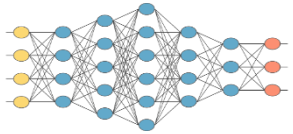
Learning with Neural Heuristics

Shah et al. ***Learning Differentiable Programs with Neural Admissible Heuristics.*** NeurIPS 2020.

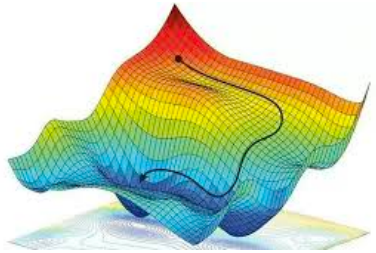
Guiding Search with Neural Relaxations



g^*



Fill hole with NN



Train parameters

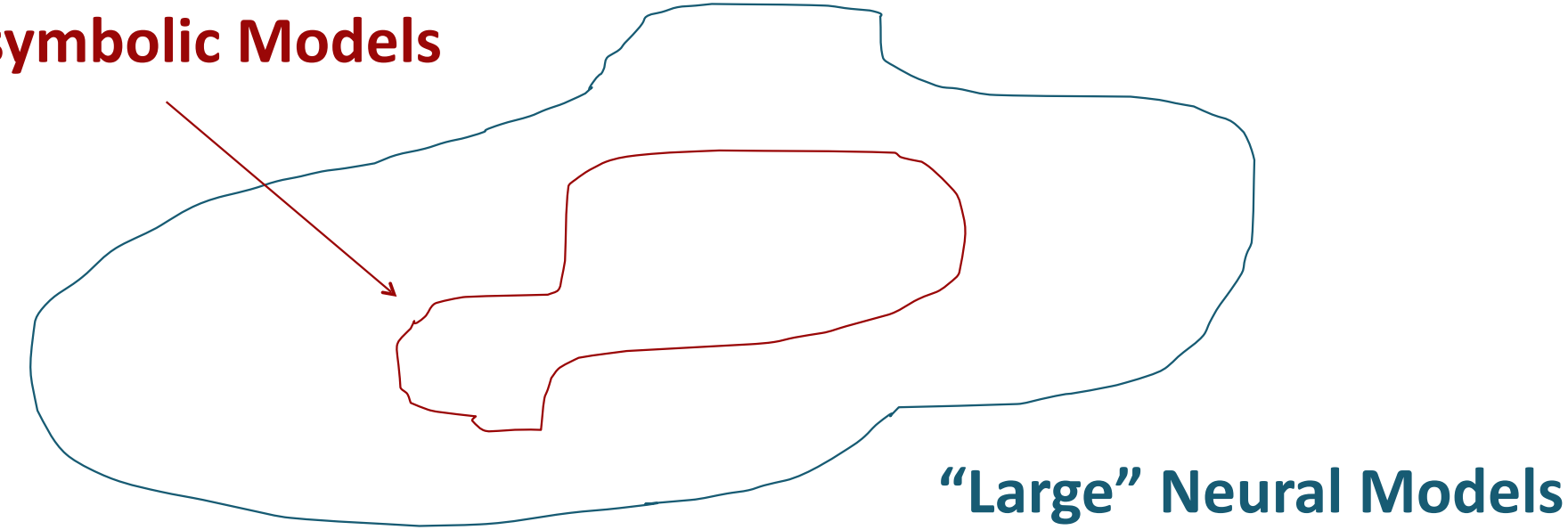
Use training loss to guide search

If a large neural network cannot fit this hole, then a neurosymbolic completion also cannot

Learning Differentiable Programs with Admissible Neural Heuristics, Ameesh Shah*, Eric Zhan*, et al., NeurIPS 2020

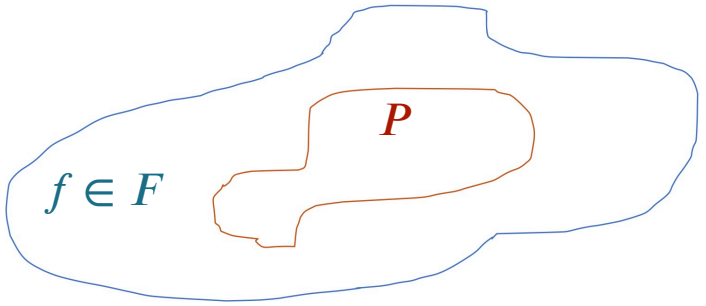
Motivating Observation/Assumption:

~~Functional Representational~~ Power



“Neural Relaxation” Every neurosymbolic model can be (approximately) represented by some “large” neural model.

Implication
(abstract form)



Large Neural

Slack due to approximation error or training ability

$$\forall P,\ \exists f \in F \text{ s.t. } d(f) \leq d(P) + \epsilon$$

Neurosymbolic

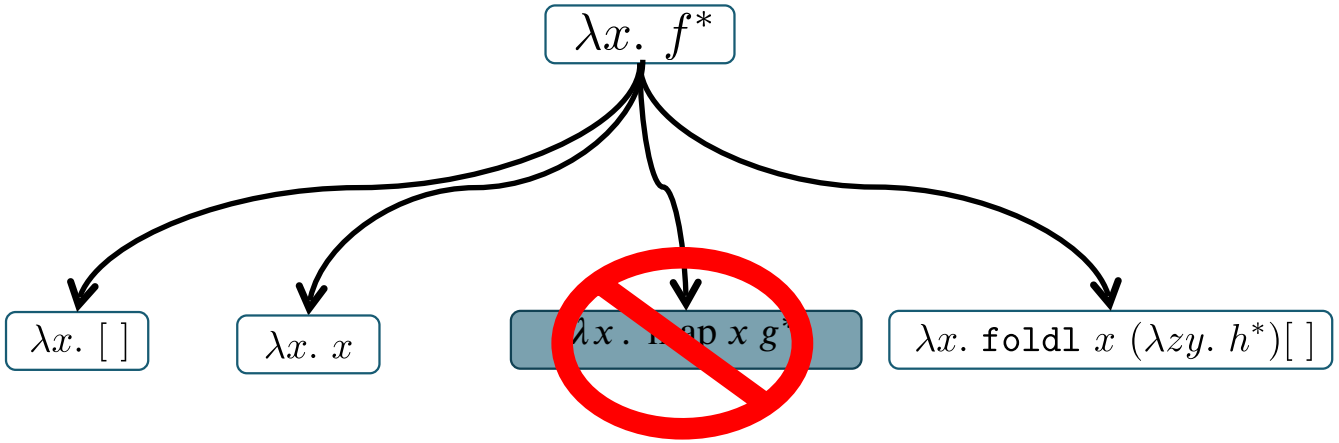
Any Cost Function

We can train an admissible heuristic!

“Neural Relaxation” Every neurosymbolic model can be (approximately) represented by some “large” neural model.

Informed Search (e.g., A*)

Use $d(P^*)$ to prune the search



Suppose:

Structural Cost

Training Loss

$s($. $\lambda x.\ map\ x\ g^*$ $) + d($. $\lambda x.\ map\ x\ g^*$ $) > s($. $\lambda x.\ x$ $) + \text{Loss}($ $\lambda x.\ x$ $)$

"Cost to Go" Heuristic

A* Search

Priority queue of current leaf nodes:

- Sorted by $s\left(P^*\right) + d(P^*)$

Pop off top program P^*

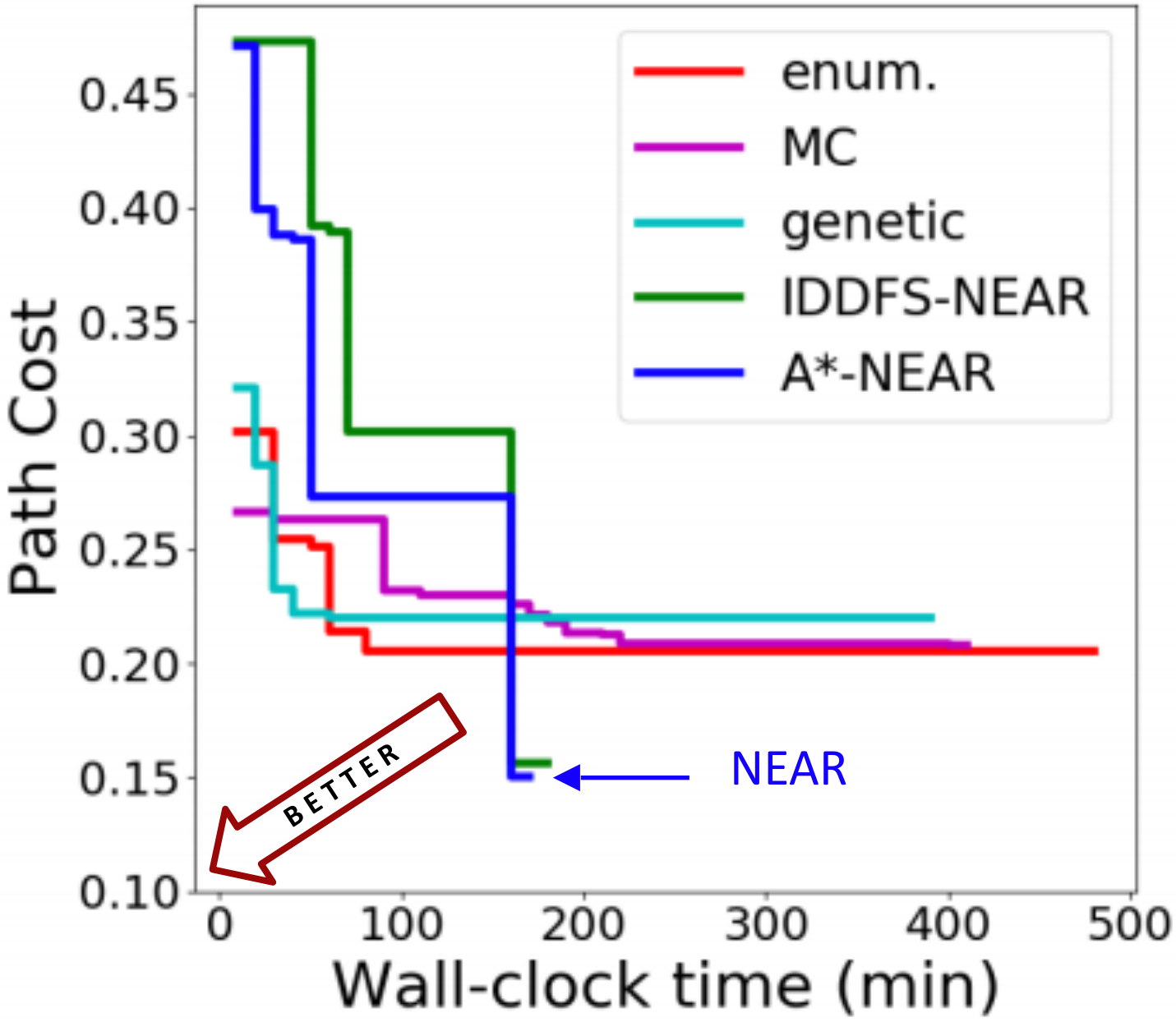
- If P^* is complete, terminate
- Else, expand P^*, add child nodes to priority queue

Lower bounds "Cost to Go"

- **Guarantee:** if $d(P^*)$ is admissible, A* will return optimal P
  - Tighter $d(P^*)$ prunes more aggressively
  - Uninformed $d\left(P^*\right)$ (e.g., always 0) => uninformed search

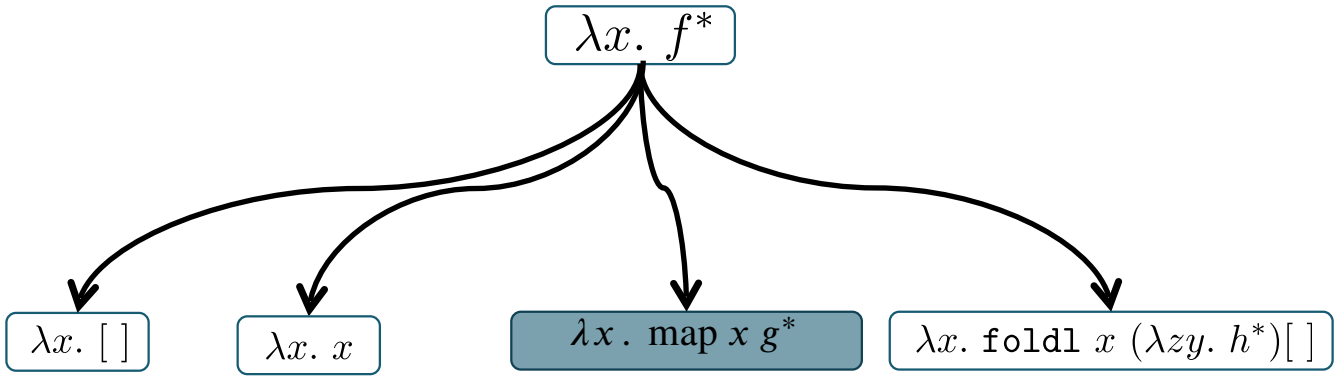
NEAR: Results

Order of magnitude speedup!

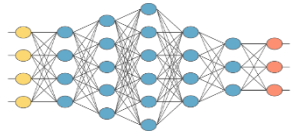


Learning Differentiable Programs with Admissible Neural Heuristics, Ameesh Shah*, Eric Zhan*, et al., NeurIPS 2020

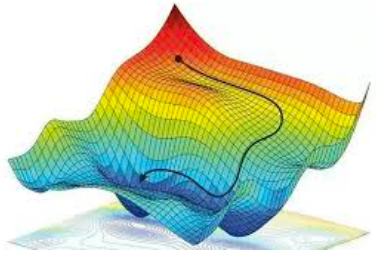
NEAR: Neural Admissible Relaxations



g^*



Fill hole with NN



Train parameters

Use training loss as admissible heuristic

If a large neural network cannot fit this hole, then a neurosymbolic completion also cannot

Learning Differentiable Programs with Admissible Neural Heuristics, Ameesh Shah*, Eric Zhan*, et al., NeurIPS 2020